

碳化硅外延层厚度的双光束和多光束干涉法测量研究

摘要

碳化硅是一种新兴的第三代半导体材料,其外延层的厚度对其性能至关重要。而红外干涉法是外延层厚度的一种无损测量方法。本文通过分析红外光束的干涉效应,建立了相应的干涉模型和厚度测量模型。采用多种优化方法,在双光束和多光束干涉场景下获得了精确的外延层厚度值。

针对问题一:构建双光束干涉模型。首先,依据 Snell 定律修正入射角的影响,推导出外延层内光程差与相位差。接着,利用 Fresnel 公式计算空气-外延层、外延层-衬底界面的反射系数,引入 Cauchy 色散模型描述折射率随波长的变化,建立外延层厚度与红外光谱反射率的模型关系。求解时通过傅里叶变换法获取干涉条纹周期并提取振荡周期,得到外延层厚度粗略的初始估计值,最后采用非线性最小二乘法拟合模型参数,为后续厚度反演提供清晰的理论框架。

针对问题二:基于问题一模型,对碳化硅晶圆片在入射角 10° 和 15° 的实验数据进行厚度反演。首先对光谱数据进行等间距重采样,裁剪 1000 cm^{-1} 以下低波数干扰区。然后对光谱利用非对称最小二乘法 (AsLS) 去趋势与 Savitzky Golay 平滑去噪。随后通过滑窗傅里叶变换筛选高信噪比有效频段 $[2500, 3700]\text{ cm}^{-1}$, 采用双角联合拟合策略共享厚度与色散参数,引入波数刻度和入射角微偏差校准量,利用随机梯度下降法进行全参拟合,最终得到 SiC 外延层厚度约 $7.604\mu\text{m}$, 残差均方根误差低于 1.14×10^{-3} 。此外,本文从残差统计特性、正态性检验、角度一致性、系统敏感性等多个维度对模型进行了可靠性检验,确保了模型的准确性与稳健性。

针对问题三:构建 Airy 多光束干涉 + Cauchy 色散模型。首先对两角度光谱进行等距重采样与轻量多项式去基线,随后采用滑窗 FFT 提取主频 f^* 与信噪比,并据此选择有效带宽 $[1170, 2170]\text{ cm}^{-1}$ 作为厚度反演的工作区间。结合相干长度判据与界面反射率锐度系数 F 的分析得到多光束相干条件,并据此判定出硅晶圆片存在显著多光束干涉,而碳化硅样品因 SiC/Si 界面反射率仅约 0.016,高阶反射贡献可忽略,沿用两光束模型即可。基于此,我们对硅样品建立 Airy 型多光束干涉 + Cauchy 色散模型,在两角度数据上联合最小二乘全参拟合,残差主峰位置复现实验条纹、幅值整体小于 0.02,最终得到硅晶圆片厚度约 $3.814\mu\text{m}$ 。

关键字: 外延层厚度测量 傅里叶变换 多光束干涉 Cauchy 色散模型 Airy 公式



一、问题重述

1.1 问题背景

碳化硅 (SiC) 作为新一代半导体材料, 因其宽禁带、高热导率和高击穿场强等特性, 被广泛应用于电力电子、轨道交通、新能源汽车等领域。外延工艺是碳化硅晶圆制造的关键环节, 而外延层的厚度直接影响器件的导通电阻、击穿电压和整体性能。因此, 如何准确测定外延层厚度, 不仅是科研中的重要课题, 也是产业界在质量控制与性能优化中的迫切需求。

与此同时, 硅材料作为传统半导体的基础, 在电子器件制造中依然占据着重要地位。对硅及碳化硅材料外延层的厚度测量与比较研究, 不仅能够为半导体产业的发展提供参考数据, 也有助于推动材料检测技术的进一步完善。

1.2 问题提出

基于上述背景, 本文建立数学模型解决下列问题:

问题一 在仅考虑外延层与衬底界面分别发生一次反射与投射所形成的两光束干涉情况下, 构建外延层厚度的数学模型。

问题二 利用问题一所建立的模型, 对碳化硅晶圆片在入射角为 10° 和 15° 时的实验数据进行厚度反演, 并对结果的准确性与可靠性进行评估。

问题三 分析多光束干涉产生的条件及其对厚度测定结果的影响, 判断硅晶圆片的实验数据是否存在多光束干涉, 并在必要时建立相应的模型与算法进行厚度求解。同时, 若碳化硅数据中存在类似影响, 提出相应的修正方法并给出校正后的厚度结果。

二、问题分析

2.1 问题一

问题一的核心是建立一个光学干涉模型, 用于描述外延层厚度与红外反射光谱之间的关系。入射光在外延层表面和衬底界面分别发生反射, 两束光在一定条件下相干叠加, 从而形成干涉条纹。条纹的周期和强度不仅依赖于外延层的厚度, 还与材料折射率和入射角有关。因此, 需要在理想化的“两光束干涉”假设下, 构建厚度与波数之间的函数关系。由于折射率随波长和载流子浓度而变化, 可引入合适的色散模型加以描述,

使得模型更贴近实际情况。通过这一模型，可以为后续的厚度估计和算法设计提供清晰的理论依据。

2.2 问题二

在已有两光束干涉模型的基础上，问题二的任务是利用实测数据对外延层厚度进行反演计算。已知条件是碳化硅晶片在两种入射角下的实测光谱曲线，关键在于如何将模型与实际数据结合，得到可靠的厚度值。

首先，需要对光谱数据进行预处理，包括去除噪声、归一化、选择合适的频段，以确保用于拟合的数据的质量。随后，根据问题一建立的模型，设计合适的算法对参数进行拟合。为了保证结果的稳定性和可信度，可以采用两步法：先通过频域分析获得厚度的初值，再利用非线性优化方法在全频段范围内精细拟合。在此过程中，充分利用入射角 10° 与 15° 的两组数据，通过联合拟合与结果对比来验证模型的适用性。最后，结合残差分析与参数灵敏度讨论，对所得厚度结果给出可靠性评估。

2.3 问题三

问题三在前两问的基础上考虑了多光束干涉的更多情况，要求分析产生多光束干涉的条件以合理选择模型解决相应问题。在建立模型之后，根据硅晶片和碳化硅晶片的实测光谱曲线，选择合适的模型，得到更精确的效果。

为此，对问题一的模型进行一些修正，将其推广到多光束情形，同时得到必要条件与精度影响。在推导获得多光束模型后，将光谱数据按问题二的流程进行预处理，通过频域分析获取初值，采用非线性优化算法对硅晶片用多光束模型进行拟合，从而评价模型的适用性，并给出可靠性评估。对碳化硅晶片，在预处理后，利用带通滤波的方法对原图进行优化，消除其他光束的影响，再利用问题二的计算方法得到厚度值。

三、模型假设

1. **厚度与面内均匀性**：假设外延层厚度 d 在光斑范围内不随波数/时间变化（面内与时间上均匀稳定）。
2. **衬底折射率平稳**：假设在选定带宽内，衬底折射率 n_2 近似平稳。SiC 样本取 $n_2 \approx 2.65$ ，Si 样本取 $n_2 \approx 3.42$ 。
3. **各向同性**：假设介质各向同性、界面理想平行，忽略粗糙、楔角影响。
4. **介质吸收率较低**：假设介质足够透明，红外光在介质表面发生折射和反射以及在介质中传播时介质对光能的吸收较低可以忽略，反射率与透射率之和为 1。

四、符号说明

本文所使用的符号及说明如表 1 所示。

表 1 符号说明

符号	含义	单位
n_i	介质对红外光折射率	—
θ_i	光线与法线的夹角	rad
δ	相位差	rad
λ	红外光波长	m
ν	单位长度内的波周期数	m^{-1}
t_{ij}, t'_{ij}	透射系数, 透射波振幅 / 入射波振幅	—
r_{ij}	反射系数, 反射波振幅 / 入射波振幅	—
T	透射率, 透射光功率 / 入射光功率	—
R	反射率, 反射光功率 / 入射光功率	—
I	光强, 衡量光的能量大小	W/m^2
$\mathbf{E}$	光矢量	—
L_c	光源相干长度	m
F	锐度系数	—
d	外延层厚度	m

五、模型建立与求解

5.1 问题一模型的建立与求解

5.1.1 两光束干涉厚度模型的建立

在红外干涉测厚实验中, 入射光在空气-外延层界面和外延层-衬底界面各发生一次反射, 两束光在探测端叠加产生干涉条纹。为描述这一现象, 我们建立“两光束干涉厚度模型”。

设空气折射率为 $n_0 = 1$ ，外延层折射率为 $n_1(\lambda)$ ，厚度为 d ，衬底折射率为 n_2 ，入射角为 θ_0 。根据 **Snell 定律**，外延层中的折射角为

$$n_0 \sin \theta_0 = n_1(\lambda) \sin \theta_1. \quad (1)$$

其中， θ_1 为外延层中的折射角。

光在外延层内的往返光程差为

$$\Delta L = 2n_1(\lambda)d \cos \theta_1, \quad (2)$$

对应的相位差为

$$\delta(\lambda) = \frac{4\pi n_1(\lambda)d \cos \theta_1}{\lambda}. \quad (3)$$

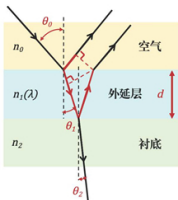


图1 两光束干涉模型的光路示意图

利用 **Fresnel 公式**，空气-外延层及外延层-衬底的反射系数分别为：

$$r_{01} = \frac{n_0 \cos \theta_0 - n_1(\lambda) \cos \theta_1}{n_0 \cos \theta_0 + n_1(\lambda) \cos \theta_1}, \quad (4)$$

$$r_{12} = \frac{n_1(\lambda) \cos \theta_1 - n_2 \cos \theta_2}{n_1(\lambda) \cos \theta_1 + n_2 \cos \theta_2}, \quad (5)$$

其中 θ_2 为在衬底中的折射角。

则总反射率为

$$R(\lambda) = \left| r_{01} + r_{12} e^{i\delta(\lambda)} \right|^2. \quad (6)$$

为刻画色散效应，引入 **Cauchy 模型**近似外延层折射率：

$$n_1(\lambda) = A + \frac{B}{\lambda^2}, \quad (7)$$

其中 A, B 为待定参数。至此，我们已建立了外延层厚度 d 与红外光谱 $R(\lambda)$ 之间的函数关系。

5.1.2 两光束干涉厚度模型的求解

由于厚度 d 出现在非线性相位函数中，难以直接解析求解，需要结合实验数据进行数值反演。求解思路如下：

首先由于干涉条纹周期与厚度的近似关系

$$\Delta\left(\frac{1}{\lambda}\right) \approx \frac{1}{2n_1 d \cos \theta_1}, \quad (8)$$

通过 FFT 分析光谱条纹周期，对厚度做出初步估计，得到厚度初值 d_0 。

接下来构造目标函数：

$$J(d, A, B) = \sum_i \left(R^{\text{obs}}(\lambda_i) - R(\lambda_i; d, A, B) \right)^2, \quad (9)$$

其中 R^{obs} 为实验数据， $R(\cdot)$ 为理论模型。

下面以 d_0 为初值，采用非线性最小二乘法拟合参数 (d, A, B) ，得到厚度估计 d^* 。

最后，对不同入射角的两组实验数据进行联合拟合，比较结果的一致性，并通过残差分析验证拟合的合理性。

5.2 问题二模型的建立与求解

5.2.1 数据预处理

在正式代入数据进行反演之前，要对数据进行预处理。为保证频域分析（FFT）与滑动窗口的有效性，需要在波数 σ 上等间距采样。将原始数据 (σ_i, R_i) 线性插值到等间距网格，步长取原始间距的中位数：

$$\Delta\sigma \approx \text{median}(\sigma_{i+1} - \sigma_i). \quad (10)$$

其中，反射率按小数参与计算： $R = R(\%) / 100$ 。

等间距重采样后得到数据如图2所示。其中 raw 代表原始数据，Uniform Sampling 代表处理之后的数据。

由图2可以看出，在原始光谱中，低波数区域（约 1000 cm^{-1} 以下），出现了明显的高反射率平台与陡降，这是由碳化硅材料的声子极化子（Reststrahlen）区引起的物理效应：当波数位于横光学声子（TO）与纵光学声子（LO）频率之间时，介电常数为负，表面反射率接近1；而在 LO 边附近（约 $970 - 1000 \text{ cm}^{-1}$ ）会出现剧烈的色散与吸收，导致光谱中出现“暴涨 暴跌”的特征。这一区域的信号主要受材料本征吸收主导，而不再体现出稳定的干涉条纹。

将该区间数据保留，会对基线估计和平滑滤波造成干扰。同时在滑窗 FFT 分析中导致主频率与信噪比的不稳定，从而降低厚度估计的可靠性。因此，我们在预处理阶段

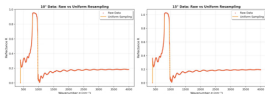


图2 等间距重采样后数据图

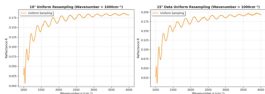


图3 裁剪低波数区域后等间距重采样图

将 1000 cm^{-1} 以下的波数段整体舍弃，仅保留 $1000\text{--}4000\text{ cm}^{-1}$ 的有效光谱进行后续分析。裁剪后的数据如图3所示。

接下来，我们对数据进行去趋势化、去噪、平滑处理。首先采用非对称最小二乘法 (AsLS) 拟合基线 $\hat{b}$ ，扣除了基线信号，有效去除了由强吸收与整体趋势引起的低频分量：

$$\min_{\hat{b}} \sum_i w_i (R_i - z_i)^2 + \lambda \sum_i (\Delta^2 z_i)^2, \quad w_i = \begin{cases} p, & R_i > z_i \\ 1-p, & R_i \leq z_i \end{cases} \quad (11)$$

其中 Δ^2 为二阶差分算子， $\lambda > 0$ 控制平滑度， $p \in (0, 1)$ 让基线贴近下包络。

随后使用 Hampel 滤波剔除尖噪，并通过 Savitzky-Golay 平滑（零相位滤波）进一步降低高频噪声。处理后得到的残差信号主要包含干涉条纹成分（见图4），为后续的频域分析和厚度估计提供了稳定的输入。需要注意的是， 1000 cm^{-1} 附近依然存在由 Reststrahlen 区边界带来的急剧起伏，因此该区域在后续分析中不予采用。

为了从干涉条纹中提取可靠的周期信息，我们采用滑动窗口快速傅里叶变换 (FFT) 的方法，对不同波数区间内的主频率与信噪比 (SNR) 进行分析。

在波数轴上选择固定宽度 $W = 300\text{ cm}^{-1}$ 的滑动窗口，依次覆盖 $1000\text{--}4000\text{ cm}^{-1}$ 的整个有效区间。对窗口内的去趋势信号 $r[n]$ 乘 Hann 窗函数 $w[n]$ ，然后计算快速傅里叶

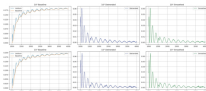


图4 去趋势化、去噪后数据图

变换得到频谱 $|Y[f]|$:

$$Y[f] = \text{FFT}[r[n] - w[n]], \quad (12)$$

将最大幅值处的频率作为该窗口的主频率 f^* :

$$f^* = \underset{f}{\operatorname{argmax}} |Y[f]|, \quad (13)$$

得到 SNR 的值:

$$\text{SNR} = \frac{A_{\text{peak}}}{\sqrt{\frac{1}{b} \sum_{n \neq f^*} A_n^2}}, \quad (14)$$

其中 A_{peak} 为主峰幅值, 邻带用于估计噪声水平。

图5给出了在 $1000\text{--}4000\text{ cm}^{-1}$ 区间内, 10° 与 15° 测量数据的滑窗 FFT 分析结果, 可以看出两角度下的主频率 f^* 基本保持稳定, 随窗口位置几乎不发生偏移, SNR 在不同波段有所差异。

在滑窗 FFT 的基础上, 我们引入了自动 band 选择准则, 确定用于厚度反演的有效频段, 我们设定“好窗”的判定条件:

1. 两个角度的信噪比需同时满足 $\text{SNR}_{10^\circ} \geq 10$, $\text{SNR}_{15^\circ} \geq 10$;
2. 两角度主频相对差满足 $\frac{|f_{10^\circ}^* - f_{15^\circ}^*|}{f_{10^\circ}^*} \leq 10\%$;
3. 将满足条件的相邻窗口合并为连续区间, 并在区间两端进行 $15\text{--}20\text{ cm}^{-1}$ 的安全裁剪;
4. 若得到多个候选区间, 则选择平均 SNR 最高且条纹周期数充足 (≥ 5 周期) 的区间作为最终 band。

筛选后将结果如图6所示, 选取 $(2500, 3700)$ 为有效频段。

在有效频段内, 对 10° 与 15° 两个角度的去趋势信号分别乘 Hann 窗并进行快速傅

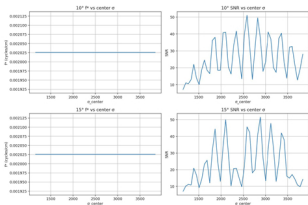


图 5 全带滑窗 FFT 结果图

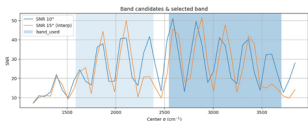


图 6 有效频段选择

里叶变换 (FFT)，提取条纹的主频 f^* 。主频与条纹周期满足关系：

$$\Delta\nu = \frac{1}{f^*}, \quad (15)$$

其中 $\Delta\nu$ 表示条纹在波数域的周期。

根据两束干涉近似条件（一次反射/透射），有：

$$\Delta\nu \approx \frac{1}{2n_2(\bar{\sigma})d \cos \theta_t}, \quad (16)$$

$$\cos \theta_t = \sqrt{1 - \left(\frac{\sin \theta}{n_2(\bar{\sigma})} \right)^2}, \quad (17)$$

其中 $n_2(\sigma)$ 为该 band 区间内的平均折射率, θ_t 为透射角。

由此可得厚度初值:

$$d_0(\theta) \approx \frac{1}{2n_{\text{eff}} \cos \theta_t \Delta\nu}. \quad (18)$$

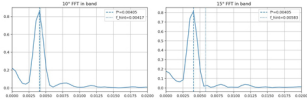


图7 执行 FFT 后的频谱结果

代入数据后得到结果如图7。10° 下估测得到厚度为 7.662 μm , 15° 下估测得到厚度为 7.682 μm , 而这相对差异小于 3%, 平均为 7.672 μm 。该厚度初值将作为后续基于 Cauchy 模型的非线性拟合的初始条件, 以保证迭代的收敛性与稳定性。

5.2.2 基于 Cauchy 折射率模型的外延层厚度反演

物理模型与参数化与校准量 设两组入射角分别为 $\theta_0^{(10)} = 10^\circ$ 与 $\theta_0^{(15)} = 15^\circ$, 对应观测数据为 $\{(\tilde{\nu}_{k,i}, R_{k,i}^{\text{obs}})\}$, 其中 $\tilde{\nu} = 1/\lambda$ 为波数, $k \in \{10, 15\}$ 标识角度。

为消除仪器与几何的系统性微偏差, 引入两类校准量:

1. 波数刻度校准 ε : 令真实波数为 $\tilde{\nu}^{\text{true}} = (1 + \varepsilon)\tilde{\nu}$, 以补偿谱仪刻度的线性误差。
2. 入射角微偏差 $\Delta\theta_k$: 令有效入射角 $\tilde{\theta}_k = \theta_0^{(k)} + \Delta\theta_k$, 以补偿装调误差。

考虑上述校准后, 两光束模型的理论反射率写为

$$R_{\text{model}}(\tilde{\nu}; d, A, B, \tilde{\theta}_k, (1 + \varepsilon)) = \mathcal{R}(\lambda = \tilde{\nu}^{-1}, d, n_1(\lambda; A, B), \tilde{\theta}_k), \quad (19)$$

其中 $\mathcal{R}(\cdot)$ 由第5.1.1节给出 (Fresnel 强度反射率 R_1, R_2 与相位差 $\delta = \frac{4\pi n_2 d \cos \theta_t}{\lambda}$ 的组合式)。

$$n(\lambda) = A + \frac{B}{\lambda^2}, \quad (20)$$

其中 A, B 为待估参数。

由 Snell 定律有 $\sin \theta_t = \sin \theta_i / n(\lambda)$, 从而 $\cos \theta_t = \sqrt{1 - \sin^2 \theta_t}$ 。

设外延层厚度为 $d [\mu\text{m}]$, 薄膜往返相位为

$$\beta(\lambda, \theta_i) = \frac{2\pi n(\lambda) d \cos \theta_t}{\lambda}. \quad (21)$$

为了将求解重点放在相位对齐（决定条纹位置、对 d 最敏感），我们采用相位优先的反射率模型：

$$\widehat{R}(\nu; \theta_i) = c_\theta + s_\theta \cos(2\beta(\lambda, \theta_i)), \quad (22)$$

其中 c_θ, s_θ 为不同入射角的幅度与偏置系数。

闭式幅值/基线标定 为抵消探测链路的幅值与基线差异，对每条曲线保留闭式线性标定：给定一组参数 $(d, A, B, \varepsilon, \Delta\theta_k)$ ，先计算理论序列 $y_{k,i} = R_{\text{model}}(\tilde{\nu}_{k,i}(1+\varepsilon); d, A, B, \tilde{\theta}_k)$ ，然后用最小二乘闭式解：

$$a_k = \frac{\text{Cov}(y_k, R_k^{\text{obs}})}{\text{Var}(y_k)}, \quad b_k = \overline{R_k^{\text{obs}}} - a_k \overline{y_k}, \quad (23)$$

得到校正后的模型输出 $\widehat{R}_{k,i} = a_k y_{k,i} + b_k$ 。这种“内环闭式、外环非线性”的结构可减少优化变量，提升稳定性。

双角联合的鲁棒目标函数 直接分别对 10° 与 15° 两组数据独立拟合厚度 d_{10} 与 d_{15} ，再取其平均作为最终结果的方法主要存在以下两大缺陷：

1. **物理一致性不足**：外延层厚度 d 在物理上应为同一常数，而独立拟合时每个角度的色散参数 (A, B) 可能向不同方向漂移，导致 d_{10}, d_{15} 不完全可比。
2. **稳健性不足**：当某一角度数据在局部波段信噪比较低或受吸收峰干扰时，独立拟合结果容易受到显著偏移，取平均并不能消除该偏差。

相比之下，**双角联合拟合** 在一个统一的目标函数下同时利用两角度的观测数据，共享厚度 d 与折射率色散参数 (A, B) ，而仅允许角度校准量 $\Delta\theta_k$ 独立变化。这种方式既保证了厚度的物理唯一性，又利用了两角度在噪声与系统误差上的互补性，从而得到更稳定、可靠的反演结果。

基于上述考虑，我们构造了带有物理先验正则的加权鲁棒最小二乘目标：

$$\min_{d, A, B, \varepsilon, \{\Delta\theta_k\}} \sum_{k \in \{10, 15\}} \sum_i w_{k,i} \rho(R_{k,i}^{\text{obs}} - \widehat{R}_{k,i}) + \underbrace{\eta_\varepsilon \varepsilon^2 + \eta_\theta \sum_k (\Delta\theta_k)^2 + \eta_A (A - A_0)^2 + \eta_B (B - B_0)^2}_{\text{物理先验正则}} \quad (24)$$

其中 $w_{k,i}$ 为数据可信度权重（在吸收段/低信噪比段取较小值）， $\rho(\cdot)$ 为鲁棒损失函数，本文选用 `soft_l1` 形式； (A_0, B_0) 为基于厚度初值 d_0 的预拟合结果， $\eta_\bullet$ 为正则权重参数。

这样构造的目标函数兼顾了“内环闭式、外环非线性”的结构：每次迭代中先用闭式解对幅值/基线进行标定，再对 $(d, A, B, \varepsilon, \Delta\theta_k)$ 进行非线性更新，从而在保证收敛稳定性的同时减少了优化变量的数量。

两阶段初始化 在确立了目标函数之后, 我们首先采用有效范围内全谱段 (约 $2500 \sim 3800 \text{ cm}^{-1}$) 进行直接拟合。对相关参数做两阶段初始化:

1. 频域初值: 由于干涉条纹主频得 $d_0, d_{0,k} = (2 \hat{f}_{0,k} \sqrt{n^2 - \sin^2 \theta_0^{(k)}})^{-1}$, 将两角合并取加权平均得 d_0 。
2. (A_0, B_0) 预拟合: 固定 $d = d_0$, 并在每条曲线内用公式 (23) 的闭式 (a_k, b_k) 对齐后, 仅对 (A, B) 做轻量最小二乘, 得到 (A_0, B_0) , 为后续全参拟合提供合理先验。

约束与求解器 为保证物理合理与数值稳定, 设置有界约束

$$d \in [1, 50] \mu\text{m}, \quad A \in [2.3, 2.8], \quad B \in [B_{\min}, B_{\max}], \quad |\varepsilon| \leq 2 \times 10^{-3}, \quad |\Delta\theta_k| \leq 0.3^\circ,$$

其中 B 的范围根据材料与谱段经验选取 (实现中用 $[-0.05, 0.20]$ 或宽松上界并配合正则拉回)。

以 $(d_0, A_0, B_0, \varepsilon=0, \Delta\theta_k=0)$ 为初值, 采用带界的 Levenberg-Marquardt/信赖域算法求解目标函数 (24), 并在启动前对 d 在 $[d_0(1 \pm 5\%)]$ 作一次一维粗搜索, 以降低局部极小风险。

拟合 我们的拟合流程如算法1所示。

Algorithm 1 两光束厚度反演

Input: $\{(\tilde{\nu}_{k,i}, R_{k,i}^{\text{obs}})\}$, $k \in \{10, 15\}$; 入射角 $\theta_0^{(k)}$; 衬底折射率 n_2

Output: 厚度 d , 色散参数 $(\hat{A}, \hat{B})$, 校准量 $\varepsilon, \hat{\Delta\theta}_k$, 置信区间

1. 频域初值: 两角分别 FFT 得主峰 $\hat{f}_{0,k}$, 合成 d_0 ;
 2. 预拟合 (A_0, B_0) : 固定 $d = d_0$, 用闭式 (a_k, b_k) 对齐后仅优化 (A, B) ;
 3. 全参联合: 以 $(d_0, A_0, B_0, 0, 0, 0)$ 为初值, 带界最小化式(24);
 4. 诊断与评估: 残差谱、双角一致性、置信区间; 必要时调整权重/带宽再迭代。
-

在引入波数刻度与入射角微偏差的校准量后, 我们首先对全谱段 ($2500 \sim 3800 \text{ cm}^{-1}$) 进行了双角联合拟合。拟合结果如图8和9所示, 厚度参数估计为

$$d = 7.619 \mu\text{m}, \quad A = 2.663, \quad B = -0.100,$$

对应的中心折射率约为 $n_1(3.23 \mu\text{m}) \approx 2.653$ 。

校准参数 ε 与 $\Delta\theta_k$ 均接近零, 表明谱仪刻度与角度设置基本准确。全谱拟合的残差均值接近零, 整体均方根误差 RMSE 分别为 1.14×10^{-3} (10°) 与 1.10×10^{-3} (15°), 说明模型能够较好地刻画主振荡区的干涉条纹。

然而, 在高波数区间 ($\tilde{\nu} > 3300 \text{ cm}^{-1}$), 残差仍呈现系统性波动, 且 RMSE 明显偏大, 提示单一 Cauchy 模型在全谱范围内的适配能力存在不足。

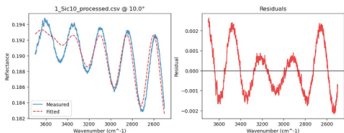


图8 10度-SiC 拟合结果

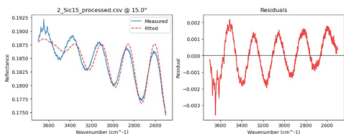


图9 15度-SiC 拟合结果

为改善这一问题,我们采用了分段拟合策略:以 $\lambda^* = 3.03 \mu\text{m}$ (对应 $\tilde{\nu} = 3300 \text{ cm}^{-1}$)为分界点,将光谱划分为低波数段与高波数段,分别估计色散参数 (A, B) ,同时要求在分界点处折射率函数 $n(\lambda)$ 及其一阶导数连续。这种处理方式能够更灵活地捕捉不同波段的色散特征,又避免了物理上的不连续性。

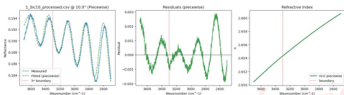


图10 10度-SiC 分段拟合结果

分段拟合的结果如图10和11所示。厚度估计 $d = 7.604 \mu\text{m}$,与全谱拟合的 $7.619 \mu\text{m}$ 基本一致(差异小于0.2%),说明分段策略并未引入厚度偏差。在分界点的连续性检查中,折射率函数值跳跃仅 $\Delta n = 2.8 \times 10^{-5}$,导数跳跃 $\Delta n' = 1.9 \times 10^{-5}$,均可忽略。

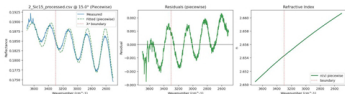


图 11 15°-SiC 分段拟合结果

更重要的是，分段方法在残差表现上优于全谱方法：

1. 全谱拟合的整体 RMSE 约为 1.14×10^{-3} ，分段拟合后在 10° 曲线上降低至 1.04×10^{-3} ，在 15° 曲线下则保持在同一数量级。
2. 在高波数区间 ($\tilde{\nu} > 3300$)，分段方法显著改善了残差，例如 10° 曲线的 RMSE 从 1.35×10^{-3} 降至 9.34×10^{-4} ，提升约 1.44 倍。

因此，分段 Cauchy 拟合在不改变厚度估计结果的前提下，有效提升了高波数段的拟合精度与残差均匀性，进一步增强了模型在全谱范围内的稳定性和可靠性。

5.2.3 结果可靠性分析

为了验证两光干涉厚度反演模型的准确性与稳健性，我们从残差统计特性、正态性检验、频域分析、角度一致性、频段稳健性、参数不确定度、系统敏感性以及合成数据回归等多个方面对模型进行了系统的可靠性检验。

1. 残差统计与正态性检验 残差直方图 12 和图 13 表明，在 10° 与 15° 两个人射角下，无论采用单角独立拟合还是双角联合拟合，残差均值接近零，分布大致对称，未表现出明显的系统性偏差。

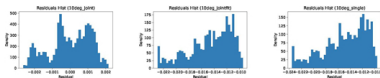


图 12 SiC 10° 联合拟合、联合回代、单角拟合残差直方图

进一步的 Q-Q 图 14 和图 15 分析显示，残差点基本落在理论正态分布直线上，拟合优度 R^2 约为 0.94–0.97，表明残差近似服从正态分布。结合 Durbin-Watson 与 Ljung-Box 检验结果，残差不存在显著自相关性，说明模型误差来源主要为随机噪声。

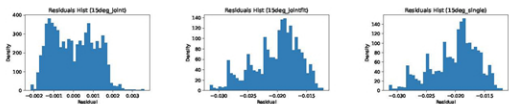


图 13 SiC 15° 联合拟合、联合回代、单角拟合残差直方图

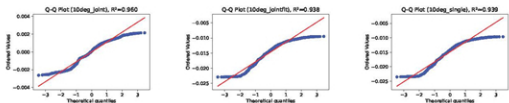


图 14 SiC 10° 联合拟合、联合回代、单角拟合残差 Q-Q 图

2. 频域残差分析 通过对残差进行快速傅里叶变换 (FFT), 从图 16和图 17中可以看出在干涉条纹主频 f^* 附近未出现显著峰值, 且相对主峰低至少 15 20 dB。残差频谱在高频区快速衰减, 未表现出周期性结构, 符合白噪声特征。联合拟合的残差谱幅度整体低于单角拟合, 进一步验证了模型拟合的充分性与可靠性。

3. 双角一致性检验 在厚度估计结果上, 由图 18可得 10° 与 15° 单角拟合得到的厚度差异小于 3%, 联合拟合结果与二者平均值的差异不超过 2%。在反射率曲线和残差分布上, 联合拟合相比单角拟合能有效降低残差波动, 并在两个角度下同时保持一致性。这说明模型在不同实验条件下具有较高的稳定性, 联合拟合策略能提升结果稳健性。

4. 频段稳健性检验 在有效波段内采用滑动窗口对不同子频段进行厚度拟合, 图 19结果显示厚度估计始终稳定在 7.59 7.64 μm 之间, 相对波动不足 1%。不同子带结果围绕全带拟合值上下波动, 未出现系统性偏移。这表明模型对频段选择不敏感, 具有良好的稳健性。

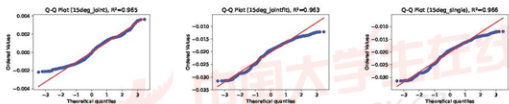


图 15 SiC 15° 联合拟合、联合回代、单角拟合残差 Q-Q 图

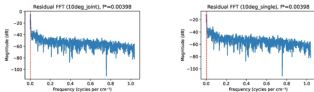


图 16 SiC 10° 联合拟合、单角拟合残差频谱图

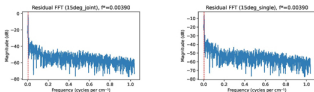


图 17 SiC 15° 联合拟合、单角拟合残差频谱图

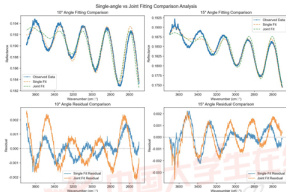


图 18 单角与联合拟合效果与残差对比

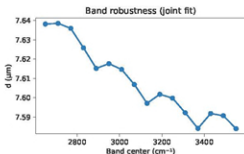


图 19 不同子频段中心下的厚度估计结果

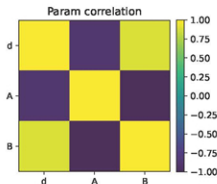


图 20 参数不确定度与相关性热图

5. 参数不确定度与相关性分析 基于最小二乘法的雅可比矩阵计算得到的协方差结果表明，厚度参数 d 的相对标准差小于 2%， A 与 B 的标准差也处于合理范围。相关性热图（图 20）显示 d 与 A 存在中等相关性， A 与 B 的相关性较高，但均低于 0.9，没有严重的共线性问题。说明参数可辨识性良好，厚度估计具有统计意义上的可信度。

6. 敏感性分析 对波数刻度误差 (ϵ) 与入射角偏差 ($\Delta\theta$) 的敏感性分析表明：在 $\pm 0.2\%$ 的波数伸缩和 $\pm 0.3^\circ$ 的角度误差范围内，厚度相对变化率分别控制在 $\pm 0.2\%$ 与 $\pm 0.04\%$ 以内。可见模型对实验系统的小扰动具有较强的鲁棒性。

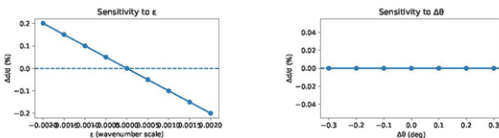


图 21 波数刻度误差、入射角偏差对厚度的敏感性

7. 合成数据回归检验 在不同信噪比条件下利用合成数据进行回归验证，图 22 结果显示随着 SNR 的提高，厚度估计偏差迅速收敛，在 $\text{SNR} \geq 20$ 时偏差保持在 0.3% 以下， $\text{SNR} = 50$ 时小于 0.2%；同时，测量精度提升显著，从约 60 ppm 降低至 10 ppm 左右。这表明模型不仅能在理想条件下准确回收真值，也能在中高信噪比实验条件下保持高稳定性。

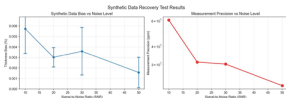


图 22 合成数据回归检验结果

通过以上多维度检验可以看出：

1. 残差统计与频域分析均支持残差服从随机噪声分布；
2. 双角一致性与频段稳健性验证了厚度反演结果的稳定性；
3. 参数不确定度分析保证了模型在统计上的可信度；
4. 敏感性与合成数据回归检验进一步证明了模型对实验误差与噪声的鲁棒性。

综上，本文建立的两光束 Cauchy 厚度反演模型在准确性、稳健性与鲁棒性方面均表现良好，能够为碳化硅外延层厚度的可靠测量提供坚实的理论与方法支持。

5.3 问题三模型的建立与求解

5.3.1 多光束干涉的必要条件

在实际实现中，光波可以在外延层界面和衬底界面产生多次反射和透射，从而产生多光束干涉。

设外延层折射率为 $n(\lambda)$ ，厚度为 d ，入射角为 θ_0 ，层内折射角为 θ （由斯涅尔定律给出）。记外表面（空气/外延层）与内表面（外延层/衬底）的复反射系数为 r_1, r_2 ，透射系数为 t_1, t_2 。各级反射光的复振幅为：

$$\mathbf{E}_1 = \mathbf{E}_0 r_1, \mathbf{E}_{m+1} = \mathbf{E}_0 t_1 t_1' r_2 e^{-i\delta} (r_1' r_2 e^{-i\delta})^{m-2}, \quad (m = 2, \dots), \quad (25)$$

其中相位推移

$$\delta(\lambda) = \frac{4\pi n(\lambda) d \cos \theta}{\lambda} = 4\pi n(\sigma) d \cos \theta \sigma, \quad (\sigma = 1/\lambda). \quad (26)$$

总场为级数之和：

$$\mathbf{E}_r = \mathbf{E}_0 r_1 + \mathbf{E}_0 t_1 t_1' r_2 e^{-i\delta} \sum_{m=2}^{\infty} (r_1' r_2 e^{-i\delta})^{m-2} = \mathbf{E}_0 \left(r_1 + \frac{(1 - r_1^2) r_2 e^{-i\delta}}{1 - r_1 r_2 e^{-i\delta}} \right). \quad (27)$$

由此得到反射率的 Airy 型表达式：

$$R(\lambda) = \frac{R_1 + R_2 + 2\sqrt{R_1 R_2} \cos \delta}{1 + R_1 R_2 + 2\sqrt{R_1 R_2} \cos \delta}, \quad R_i = |r_i|^2. \quad (28)$$

该式表明：条纹的相位由 δ 决定，线形与峰锐度由 R_1, R_2 决定。

故产生多光束干涉需要满足以下条件：

1. 相干长度条件：光源相干长度 L_c 应大于往返光程差，

$$L_c \gtrsim 2nd \cos \theta. \quad (29)$$

2. 反射率条件：界面反射率不能过小，需保证 $|r_1 r_2| \ll 1$ 。锐度系数

$$F = \frac{4\sqrt{R_1 R_2}}{(1 - \sqrt{R_1 R_2})^2} \quad (30)$$

不应太小。

3. 低吸收条件：外延层吸收系数 α 较小，使得 $e^{-2\alpha d \cos \theta}$ 不过小，否则高阶光被显著削弱。
4. 几何与分辨条件：入射光束平行性好，样品楔角小，仪器分辨率优于条纹间隔。

5.3.2 多光束干涉对厚度计算精度可能的影响

设条纹相位为 $\delta = 4\pi n d \cos \theta / \lambda_s$ 。

1. 在理想对称、低损耗条件下，极值近似满足 $\delta = 2\pi m$ ，条纹间隔

$$\Delta \sigma \approx \frac{1}{2nd \cos \theta} \quad (31)$$

与两光束模型一致。因此，多光束干涉并不显著改变条纹的基本周期。

2. 多光束干涉会使条纹更尖锐、对比度增强。如果仍使用两光束正弦模型拟合，会出现模型失配，导致残差具有结构性，从而引入系统偏差。
3. 当 $R_1 \neq R_2$ 或外延层存在吸收时，极值位置发生微小漂移，若用两光束近似会进一步放大厚度计算误差。
4. 在傅里叶域，多光束干涉会引入谐波分量，若忽略处理，可能造成厚度估计偏差。

综上，多光束干涉的存在会影响厚度计算的精度，需要在建模与拟合过程中予以考虑。

5.3.3 硅晶圆数据的预处理与多光束干涉分析

在对附件 3（入射角 10° ）与附件 4（入射角 15° ）的硅晶圆片实验数据进行分析之前，需要进行合理的预处理，以保证后续频谱分析与厚度估算的可靠性。

数据统一化与波段裁剪 原始数据的波数轴 σ （单位： cm^{-1} ）并非严格等间距，反射率列单位为%。首先按波数升序排序，将反射率归一化到 $[0, 1]$ ，并采用线性插值将数据统一到等间距网格

$$\{\sigma_k\}_{k=0}^{N-1}, \quad \sigma_{k+1} - \sigma_k = \Delta \sigma \approx \text{median}\{\Delta \sigma_{\text{mw}}\}. \quad (32)$$

统一采样后的效果见图 23：重采样曲线与原始散点高度重合，保证了后续 FFT 的采样假设。

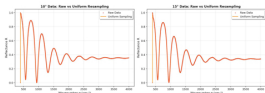


图 23 硅片反射率光谱：原始散点 vs. 等间距重采样（左： 10° ；右： 15° ）。

考虑到低波数段噪声与仪器畸变相对明显，我们仅保留 $500 \sim 4000 \text{ cm}^{-1}$ 的范围作为有效工作带。

基线校正 我们采用轻量化的多项式拟合去基线方法：对归一化反射率 $R(\sigma)$ 与波数 σ 进行一次或二次多项式拟合

$$R_{\text{trend}}(\sigma) \approx a_0 + a_1\sigma \quad (\text{或 } a_0 + a_1\sigma + a_2\sigma^2), \quad (33)$$

再将其从原始曲线上扣除，得到去趋势后的干涉信号

$$R_{\text{detrended}}(\sigma) = R(\sigma) - R_{\text{trend}}(\sigma). \quad (34)$$

与复杂的基线校正算法相比，该方法简洁且能够保留干涉条纹的相位信息，避免过度滤波导致条纹畸变。

图 24 给出了入射角 10° 与 15° 数据的去基线效果，可以看到条纹的幅度更加突出，便于后续 FFT 提取主频。

滑窗 FFT 与信噪比评估 为验证条纹周期的稳定性与预处理效果，采用汉宁窗的滑动窗口快速傅里叶变换 (Sliding-FFT) 提取局部主频 f^* 与信噪比 SNR。窗口宽度 W 取 1000 cm^{-1} (覆盖 ≥ 4 个条纹周期)，步长 150 cm^{-1} 。对每个窗信号 x ，先行去均值并乘以窗函数 w ，计算

$$Y = |\text{FFT}\{w \cdot (x - \bar{x})\}|, \quad (35)$$

主峰幅度记为 A_{pk} ，噪声幅度取主峰左右排除若干频点后的均方根，得到 $\text{SNR} = A_{pk}/\text{RMS}(\text{noise})$ 。

图 25 与图 26 显示，预处理后提取的 $f^*(\sigma)$ 更加平滑稳定，低频伪峰显著下降，SNR 分布合理。

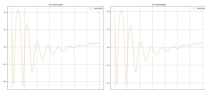


图 24 经基线校正后的硅片反射率光谱

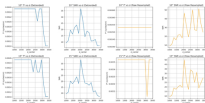


图 25 窗谱 FFT：去基线 vs. 仅重采样的 f^* 与 SNR 对比

信噪比驱动的有效分析窗选择 为获得稳健的厚度估算区间，我们以窗谱 FFT 的 SNR 与频率稳定性为准则，选择 $\text{SNR} \geq 15$ 且 f^* 偏离其带内中位数不超过 3% 的连续区间作为有效窗。两角度数据均指向 $[1170, 2170] \text{ cm}^{-1}$ 为最优区间（见图 27）。

多光束干涉的实证判定 根据第 5.3.1 节给出的多光束干涉必要条件，结合硅空气折射率差较大及实验光源条件，可知样品具备产生多光束干涉的物理前提。对附件 3.8.4 的实测数据，证验如下：

1. 条纹可见度与尖锐度：在 $600 \sim 1500 \text{ cm}^{-1}$ 区间，峰谷对比度很高，条纹呈现 Airy 线形的尖峰特征，显著强于两光束近似下的正弦型条纹。
2. 频域特征：窗谱 FFT 以单一主频为主，但在主频两侧可见次级拖尾。窗谱 FFT 的幅度在主频附近存在可观察的谐波成分，符合多次反射叠加对线形的修饰。

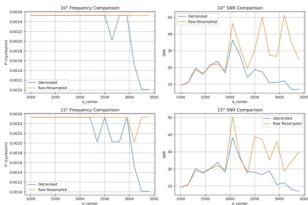


图 26 局部主频与 SNR 的叠图比较

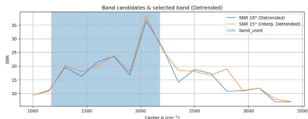


图 27 基于滑窗 FFT 的信噪比与频率稳定性选带

3. 跨角度一致性: 将局部主频按 $\cos \theta$ 归一化后, 10° 与 15° 两角度在所选带内一致性良好 (相对差异小于数个百分点), 说明条纹周期受几何光程主导, 而线形差异来自多光束精度变化。

基于以上图像特征与频域证据, 可判定附件 3 与附件 4 的硅片数据存在显著的多光束干涉效应。

5.3.4 多光束干涉厚度模型的建立

下面, 我们建立多光束干涉下的反射率模型。设所有反射光振幅叠加为:

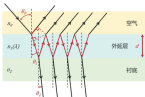


图 28 两光束干涉模型的光路示意图

$$E_r = E_0(r_1 + t_2 r_1' r_2 e^{-i\delta} + t_2 r_2' r_1' r_2^2 e^{-2i\delta} + \dots + t_2 r_1' r_1^{m-1} r_2^{m-1} e^{-i(m-1)\delta} + \dots), \quad (36)$$

其中 $\delta = 2\pi n_2 d \cos \theta_2$ 为相位差。

由于 $|r_1 r_2| < 1$ ，上述级数收敛，可视为几何级数求和，得到

$$E_r = E_0 \left(r_1 + \frac{(1 - r_1^2) r_2 e^{-i\delta}}{1 - r_1 r_2 e^{-i\delta}} \right). \quad (37)$$

由此可得反射率

$$R = \frac{|E_r|^2}{|E_0|^2} = \frac{r_1^2 + r_2^2 + 2r_1 r_2 \cos \delta}{1 + r_1^2 r_2^2 + 2r_1 r_2 \cos \delta} = \frac{(r_1 + r_2)^2 - 4r_1 r_2 \sin^2 \frac{\delta}{2}}{(1 + r_1 r_2)^2 - 4r_1 r_2 \sin^2 \frac{\delta}{2}}. \quad (38)$$

这就是典型的 *Airy* 干涉公式，与 *Fabry-Pérot* 干涉腔形式一致。为便于分析，记 $R_1 = r_1^2$ ， $R_2 = r_2^2$ ，则有

$$R = \frac{R_{\max} - F \sin^2 \frac{\delta}{2}}{1 - F \sin^2 \frac{\delta}{2}}, \quad (39)$$

其中 F 称为锐度系数，这里定义为

$$F = \frac{4\sqrt{R_1 R_2}}{(1 + \sqrt{R_1 R_2})^2}. \quad (40)$$

并有

$$R_{\max} = \left(\frac{\sqrt{R_1} + \sqrt{R_2}}{1 + \sqrt{R_1 R_2}} \right)^2, \quad R_{\min} = \left(\frac{\sqrt{R_1} - \sqrt{R_2}}{1 - \sqrt{R_1 R_2}} \right)^2. \quad (41)$$

界面反射率 R_1 、 R_2 可由菲涅耳公式计算：

$$R_0 = \frac{1}{2} \left(\frac{\sin^2(\theta_0 - \theta_1)}{\sin^2(\theta_0 + \theta_1)} + \frac{\tan^2(\theta_0 - \theta_1)}{\tan^2(\theta_0 + \theta_1)} \right), \quad (42)$$

$$R_2 = \frac{1}{2} \left(\frac{\sin^2(\theta_1 - \theta_2)}{\sin^2(\theta_1 + \theta_2)} + \frac{\tan^2(\theta_1 - \theta_2)}{\tan^2(\theta_1 + \theta_2)} \right). \quad (43)$$

由以上推导可见,多光束干涉下的反射率由相位 δ 控制。条纹极大值与极小值分别对应 $R_{\max}$ 和 $R_{\min}$, 条纹锐度取决于精细度系数 F 。当 F 较大时,条纹峰值尖锐,厚度分辨率提高,但若仍采用两光束近似拟合,会产生系统偏差;当 F 较小时,多光束效应减弱,可近似为两光束模型。

值得注意的是,厚度 d 与相位 δ 的关系依然保持周期性:

$$\delta = \frac{4\pi n d \cos \theta}{\lambda} \Rightarrow \Delta \sigma \approx \frac{1}{2nd \cos \theta}. \quad (44)$$

因此,厚度的主周期估算公式与两光束情况一致,但由于 Airy 线形导致的极值微移和非对称性,若直接使用两光束模型,将低估计算的不确定度,并可能引入 1% ~ 2% 的系统偏差。为获得高精度厚度,应在建模和拟合中采用 Airy 型多光束模型。

5.3.5 多光束干涉厚度模型的求解

Step1 数据预处理 数据预处理的工作已经在 5.3.3 完成,此书不再赘述。

Step2 FFT 粗估厚度 在多光束干涉的情形下,干涉条纹依旧表现为近似周期性。我们仍然对预处理后的反射率光谱 $R(\nu)$ (ν 为波数,单位 cm^{-1}) 进行快速傅里叶变换 (FFT),从频谱中提取主频率 f_0 。该频率与光程差的关系为

$$d_0 \approx \frac{f_0}{2n_{\text{eff}} \cos \theta}, \quad (45)$$

其中 n_{eff} 为外延层的有效折射率, θ 为入射角。

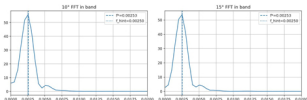


图 29 执行 FFT 后的频谱结果

代入数据后得到结果如图 29。10° 下估测得到厚度为 3.675 μm , 15° 下估测得到厚度为 3.681 μm , 平均为 3.678 μm 。该厚度初值将作为后续基于 Cauchy 模型的非线性拟合的初始条件,以保证迭代的收敛性与稳定性。

Step3 多光电子干涉-Coschy 模型拟合 为了同时考虑非光电子干涉效应与折射率色散, 我们采用基于 Coschy 模型的 Aky 拟合, 折射率表示为

$$n(\lambda) = A + \frac{B}{\lambda^2} \quad (44)$$

其中 A, B 为色散参数。

将其代入非光电子干涉反射率公式, 可得:

$$R(x, \lambda; A, B, \alpha, \Delta\theta_1, \Delta\theta_2) = f(x, \lambda; n(\lambda), \theta + \Delta\theta_1, x), \quad (45)$$

其中 x 为外延层厚度, x 表示波数厚度的微小修正, $\Delta\theta_1, \Delta\theta_2$ 分别为两个反射面的相位修正。

在数值实现上, 我们采用最小二乘法, 针对每条曲线上引入线性拟合因子 a_0, a_1 , 通过响应函数

$$res = \sum_i [a_0 R_{fit}(x) + a_1 - R_i^{exp}(x)]^2 \quad (46)$$

作为目标函数进行迭代求解。

为避免过拟合, 我们优化中引入了对 $A, B, \alpha, \Delta\theta$ 的正则化约束, 避免参数偏离物理合理区间。

在 10° 与 1° 两个人射角数据拟合条件下, 得到的拟合结果为:

$$\hat{\lambda} = 3.814 \mu\text{m}, \quad \hat{A} = 3.445, \quad \hat{B} = 0.003.$$

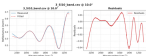


图 30 18 线拟合结果

从拟合曲线曲线 (见图 30 和图 31) 可以看出, 曲线整体偏差小于 0.02, 余值主峰和谷值位置均能准确发现, 验证了模型的可变性。

4.2.4 响应函数拟合与多光电子干涉的考察

对于研究能带结构数据, 我们利用 FDTD 方法对 10° 与 1° 两个人射角的光谱进行拟合分析, 图 31 结果展示两曲线下的主峰 λ' 在 3.80-3.9 左右, 主峰率也单一, 且 $\lambda' \approx \hat{\lambda}$

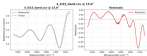


图 31 15°-60°拟合结果

在两个角度下基本一致，说明干涉条纹的周期性由几何光程主导，本实验显著的峰峰或谷谷。

进一步结合界面反射率的物理估算，空气/NaCl 界面的反射率约为

$$R_0 = \left(\frac{n_{\text{NaCl}} - 1}{n_{\text{NaCl}} + 1} \right)^2 \approx 0.03, \quad (39)$$

而 NaCl/Na 界面的反射率约为

$$R_1 = \left(\frac{n_{\text{Na}} - n_{\text{NaCl}}}{n_{\text{Na}} + n_{\text{NaCl}}} \right)^2 \approx 0.003, \quad (40)$$

两束光透射的周期因子为 $R_0 R_1 \approx 1.1 \times 10^{-4}$ ，其中高阶项迅速衰减，因此该结构的透射谱强度 (Intensity) 较弱，多次反射的贡献远小于主干涉项。

从实验结果来看，图 19 和图 11 显示基于两束光干涉模型拟合曲线整体在 $10^{-3} \sim 10^{-2}$ 量级，无明显周期性波动或次要效应，进一步表明高阶多次光效应应在 NaCl 测量中可以忽略。

在本文我们已建立了多光束 Airy 型干涉模型，并给出了相应的参数计算方法。我们也基于 Coody 的拟合程序 (附件 14) 的优化拟合函数进行了非线性最小二乘拟合，参数包括厚度 d 、Coody 系数 A 和 B 、波数修正因子 r 以及入射角修正 $\Delta\theta_1, \Delta\theta_2$ 。

拟合结果如下：

$$d = 7.620 \mu\text{m}, \quad A = 1.60000, \quad B = 1.1 \times 10^{-4},$$

$$r = 1.05 \times 10^{-10}, \quad \Delta\theta_1 = 0^\circ, \quad \Delta\theta_2 = 0^\circ.$$

优化过程由 12 次迭代后收敛，初始拟合函数为 1.48×10^{-2} ，最终代价值降低至 1.48×10^{-3} ，收敛率为 10% 附近。

从图 10 和图 10 可以看到，该结果与两束光模型下得到的模型拟合曲线基本一致，且其分布范围与点测物理量一致。

综上，优化拟合结果与实验结果在主要参数范围内两束光干涉，多次光效应不明显。两束干涉模型已足以解释实验数据并保证厚度拟合的精度。

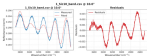


图 32 多先电干涉模型下 10 阶模拟合结果

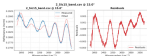


图 33 多先电干涉模型下 14 阶模拟合结果

六、模型评价

6.1 模型优点

1. 理论基础性强：多先电干涉模型和多先电干涉模型均是基于电磁场理论的科学推导，具有坚实的理论基础，理论原理与逻辑完备，模型拟合精度较高，使得的结果具有很好的可靠性。
2. 模型普适性：多先电干涉模型适用于各种不同结构的模型，可用于解决各种不同类型的干涉问题，具有较大的适用范围。
3. 模型设计简单易懂：多先电干涉模型的设计思路清晰，有效结合了全波的计算，结果可靠，由此模型得到的干涉结果精确，为物理量拟合拟合降低误差，增加计算效率。

6.2 模型缺点

1. 模型模型复杂度较高：多先电干涉模型具有较多的参数以及复杂的表达式，因此模型的复杂度较高，在求解时计算复杂度较高，运行成本较高，在应用范围可能有所欠缺，而

为了降低优化算法的成本,容易导致拟合时有较强的初值敏感性,且容易陷入局部最优,因此需要设定较好的初始值,并做好残差分析和多参数交叉验证。

2. **未考虑外延层对光的吸收:**两模型均假设外延层足够透明,对光能的吸收率较小以至于可以忽略。在本征吸收频率附近时两模型已不再适用,需考虑原子光谱等更加复杂的修正。

参考文献

- [1] 全国半导体设备和材料标准化技术委员会 (SAC/TC 203), 全国半导体设备和材料标准化技术委员会材料分技术委员会 (SAC/TC 203/SC 2). 碳化硅外延层厚度的测试 红外反射法: GB/T 42905-2023[S]. 中国标准出版社, 2023.
- [2] 全国半导体设备和材料标准化技术委员会材料分技术委员会 (SAC/TC 203/SC 2). 重掺杂衬底上轻掺杂硅外延层厚度的红外反射测量方法: GB/T 14847-2010[S]. 中国标准出版社, 2010.
- [3] 赵凯华, 钟锡华. 光学: 上册 [M]. 北京: 北京大学出版社, 1984.
- [4] 郭小花, 张明霞. 多光束干涉的扩展研究 [J]. 天水师范学院学报, 2016, 36(05): 13-14.
- [5] 郭硕鸿. 电动力学 [M]. 北京: 高等教育出版社, 2000. 143 - 147.
- [6] 程佩青. 数字信号处理教程 [M]. 北京: 清华大学出版社, 2017
- [7] DeepSeek. DeepSeek-R1-0528, 深度求索 (DeepSeek), 2025-09-05
- [8] ChatGPT, GPT-4o, OpenAI, 2025-09-06



附录 A SiC 外延层厚度分析源代码 1

```
import os, json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import sparse
from scipy.signal import savgol_filter
from scipy.signal import windows
from numpy.fft import rfft, rfftfreq

# === Phase 1: load & preprocess data ===
plt.rcParams["figure.figsize"] = (8, 4.5)
plt.rcParams["figure.dpi"] = 140
plt.rcParams["axes.grid"] = True

LOW_CUT = 1000

os.makedirs("./figs", exist_ok=True)
os.makedirs("./preprocessed", exist_ok=True)

def clip_low(s, y, low=LOW_CUT):
    m = s >= low
    return s[m], y[m]

def load_xlsx(path):
    df = pd.read_excel(path, header=0)
    sigma = pd.to_numeric(df.iloc[:, 0],
                          errors="coerce").dropna().to_numpy(float)
    R_pct = pd.to_numeric(df.iloc[:, 1],
                          errors="coerce").dropna().to_numpy(float)

    R = R_pct / 100.0 if np.nanmax(R_pct) > 1.5 else R_pct

    # sort
    order = np.argsort(sigma)
    sigma, R = sigma[order], R[order]

    # normalization
    R_min, R_max = float(np.min(R)), float(np.max(R))
    R_norm = (R - R_min) / (R_max - R_min + 1e-12)

    return sigma, R_norm
```

```

def make_uniform_sigma(sigma, R, step=None):
    idx = np.argsort(sigma)
    sigma = sigma[idx]
    R = R[idx]
    diffs = np.diff(sigma)
    if step is None:
        step = float(np.median(diffs))
    sigma_u = np.arange(sigma.min(), sigma.max() + step / 2, step)
    R_u = np.interp(sigma_u, sigma, R)
    return sigma_u, R_u, step

# load data
signal0_raw, R10_raw = load_xlsx("./1_Sic10.xlsx")
signal15_raw, R15_raw = load_xlsx("./2_Sic15.xlsx")

signal0_u, R10_u, dstep10 = make_uniform_sigma(signal0_raw, R10_raw)
signal15_u, R15_u, dstep15 = make_uniform_sigma(signal15_raw, R15_raw)

fig, axs = plt.subplots(1, 2, figsize=(14, 5))

# left -> angle 10°
axs[0].scatter(signal0_raw,
                R10_raw,
                s=8,
                alpha=0.7,
                label="Raw Data",
                color='red',
                edgecolors='none')
axs[0].plot(signal0_u,
            R10_u,
            lw=2,
            label="Uniform Sampling",
            color='darkorange',
            alpha=0.8)
axs[0].set_title("10° Data: Raw vs Uniform Resampling",
                fontsize=12,
                fontweight='bold')
axs[0].set_xlabel("Wavenumber (cm-1)", fontsize=11)
axs[0].set_ylabel("Reflectance R", fontsize=11)
axs[0].legend(frameon=True, fancybox=True, shadow=True)
axs[0].grid(True, alpha=0.3)

# right -> angle 15°
axs[1].scatter(signal15_raw,
                R15_raw,

```

```

        s=8,
        alpha=0.7,
        label="Raw Data",
        color='red',
        edgecolors='none')
    axs[1].plot(signal15_u,
        R15_u,
        lw=2,
        label="Uniform Sampling",
        color='darkorange',
        alpha=0.8)
    axs[1].set_title("15° Data: Raw vs Uniform Resampling",
        fontsize=12,
        fontweight='bold')
    axs[1].set_xlabel("Wavenumber (cm⁻¹)", fontsize=11)
    axs[1].set_ylabel("Reflectance R", fontsize=11)
    axs[1].legend(frameon=True, fancybox=True, shadow=True)
    axs[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("../figs/raw_vs_uniform.png")
plt.show()

signal10_u, R10_u = clip_low(signal10_u, R10_u)
signal15_u, R15_u = clip_low(signal15_u, R15_u)

# enhanced plot
fig, axs = plt.subplots(1, 2, figsize=(14, 5))

# left -> angle 10°
axs[0].plot(signal10_u,
    R10_u,
    lw=2,
    label="Uniform Sampling",
    color='darkorange',
    alpha=0.8)
axs[0].set_title("10° Uniform Resampling (Wavenumber > 1000cm⁻¹)",
    fontsize=12,
    fontweight='bold')
axs[0].set_xlabel("Wavenumber (cm⁻¹)", fontsize=11)
axs[0].set_ylabel("Reflectance R", fontsize=11)
axs[0].legend(frameon=True, fancybox=True, shadow=True)
axs[0].grid(True, alpha=0.3)

# right -> angle 15°
axs[1].plot(signal15_u,
    R15_u,

```

```

        lw=2,
        label="Uniform Sampling",
        color='darkorange',
        alpha=0.8)
axs[1].set_title("15° Data Uniform Resampling (Wavenumber > 1000cm-1)",
                fontsize=12,
                fontweight='bold')
axs[1].set_xlabel("Wavenumber (cm-1)", fontsize=11)
axs[1].set_ylabel("Reflectance R", fontsize=11)
axs[1].legend(frameon=True, fancybox=True, shadow=True)
axs[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("./figs/uniform_above1000.png")
plt.show()

# === Phase 2: detrending & smoothing ===

def baseline_asls(y, lam=1e5, p=0.001, niter=10):
    L = len(y)
    D = sparse.diags([1, -2, 1], [0, -1, -2], shape=(L - 2, L))
    D = D.T @ D
    w = np.ones(L)
    for _ in range(niter):
        W = sparse.diags(w, 0, shape=(L, L))
        Z = W + lam * D
        z = sparse.linalg.spsolve(Z, w * y)
        w = p * (y > z) + (1 - p) * (y <= z)
    return z

def hampel_filter(x, k=3, t=3.0):
    x = x.copy()
    n = len(x)
    for i in range(n):
        lo, hi = max(0, i - k), min(n, i + k + 1)
        win = x[lo:hi]
        med = np.median(win)
        mad = np.median(np.abs(win - med)) + 1e-12
        if np.abs(x[i] - med) > t * 1.4826 * mad:
            x[i] = med
    return x

def detrend_and_smooth(R_u, sg_win=11, sg_poly=2, lam=1e5, p=0.001):
    R1 = hampel_filter(R_u, k=3, t=3.0)

```

```

base = baseline_asls(R1, lam=lam, p=p, niter=10)
r_det = R1 - base
sg_win = sg_win if sg_win % 2 == 1 else sg_win + 1
r_smooth = savgol_filter(r_det, sg_win, sg_poly, mode="interp")
return r_det, r_smooth, base

# detrending & smoothing
r10_det, r10_smooth, base10 = detrend_and_smooth(R10_u,
                                                sg_win=11,
                                                sg_poly=2,
                                                lam=1e5,
                                                p=0.001)
r15_det, r15_smooth, base15 = detrend_and_smooth(R15_u,
                                                sg_win=11,
                                                sg_poly=2,
                                                lam=1e5,
                                                p=0.001)

# plot
fig, axs = plt.subplots(2, 3, figsize=(18, 8))
axs[0, 0].plot(signal0_u, R10_u, lw=1.0, label="Uniform", color='red')
axs[0, 0].plot(signal0_u, base10, lw=1.0, label="Baseline", color='darkorange')
axs[0, 0].set_title("10° Baseline")
axs[0, 0].legend()

axs[0, 1].plot(signal0_u, r10_det, lw=1.0, label="Detrended", color='red')
axs[0, 1].set_title("10° Detrended")
axs[0, 1].legend()

axs[0, 2].plot(signal0_u,
              r10_smooth,
              lw=1.0,
              label="Smoothed",
              color='darkorange')
axs[0, 2].set_title("10° Smoothed")
axs[0, 2].legend()

axs[1, 0].plot(signal5_u, R15_u, lw=1.0, label="Uniform", color='red')
axs[1, 0].plot(signal5_u, base15, lw=1.0, label="Baseline", color='darkorange')
axs[1, 0].set_title("15° Baseline")
axs[1, 0].legend()

axs[1, 1].plot(signal5_u, r15_det, lw=1.0, label="Detrended", color='red')
axs[1, 1].set_title("15° Detrended")
axs[1, 1].legend()

```

```

    axes[1, 2].plot(signal15_u,
                    r15_smooth,
                    lw=1.0,
                    label="Smoothed",
                    color='darkorange')
axes[1, 2].set_title("15° Smoothed")
axes[1, 2].legend()

plt.tight_layout()
plt.savefig("../figs/detrend_smooth.png")
plt.show()

# save data
df_10 = pd.DataFrame({
    'wavenumber_cm-1': signal10_u,
    'reflectance_uniform': R10_u,
    'baseline': base10,
    'detrended': r10_det,
    'smoothed': r10_smooth
})
df_10.to_csv('../preprocessed/1_Sic10_processed.csv', index=False)

df_15 = pd.DataFrame({
    'wavenumber_cm-1': signal15_u,
    'reflectance_uniform': R15_u,
    'baseline': base15,
    'detrended': r15_det,
    'smoothed': r15_smooth
})
df_15.to_csv('../preprocessed/2_Sic15_processed.csv', index=False)

# === Phase 3: Sliding FFT ===

def sliding_fft_metrics(sigma,
                        y,
                        win_width_cm=300,
                        step_cm=60,
                        lowfreq_cut_bins=1,
                        zero_pad=True):
    ds = float(np.median(np.diff(sigma)))
    W = int(round(win_width_cm / ds))
    S = int(round(step_cm / ds))
    W = W if W % 2 == 1 else W + 1
    centers, fstar, snr = [], [], []
    i = 0
    while i + W <= len(y):

```

```

seg = y[i:i + W]
sig_seg = sigma[i:i + W]
w = windows.hann(W)
x = seg * w
nfft = 1 << (W - 1).bit_length() if zero_pad else W
Y = np.abs(rfft(x, n=nfft))
F = rfftfreq(nfft, d=ds)
lo = max(1, lowfreq_cut_bins)
pk_idx = lo + np.argmax(Y[lo:])
fpk = F[pk_idx]
amp = Y[pk_idx]
nb = np.r_[max(lo, pk_idx - 50):max(lo, pk_idx - 3),
          min(pk_idx + 3, len(Y)):min(pk_idx + 50, len(Y))]
noise = np.sqrt(np.mean(Y[nb]**2)) + 1e-12
centers.append(sig_seg[W // 2])
fstar.append(fpk)
snr.append(amp / noise)
i += S
return np.array(centers), np.array(fstar), np.array(snr)

```

```

cent10, f10, s10 = sliding_fft_metrics(sigma10_u, r10_smooth)
cent15, f15, s15 = sliding_fft_metrics(sigma15_u, r15_smooth)

```

```

fig, axs = plt.subplots(2, 2, figsize=(12, 8))
axs[0, 0].plot(cent10, f10)
axs[0, 0].set_title("10° f* vs center ")
axs[0, 0].set_xlabel("_center")
axs[0, 0].set_ylabel("f* (cycles/cm)")
axs[0, 1].plot(cent10, s10)
axs[0, 1].set_title("10° SNR vs center ")
axs[0, 1].set_xlabel("_center")
axs[0, 1].set_ylabel("SNR")
axs[1, 0].plot(cent15, f15)
axs[1, 0].set_title("15° f* vs center ")
axs[1, 0].set_xlabel("_center")
axs[1, 0].set_ylabel("f* (cycles/cm)")
axs[1, 1].plot(cent15, s15)
axs[1, 1].set_title("15° SNR vs center ")
axs[1, 1].set_xlabel("_center")
axs[1, 1].set_ylabel("SNR")
plt.tight_layout()
plt.savefig("../figs/sliding_fft.png")
plt.show()

```

```

# === Phase 4: Automatic Band Selection ===

```

```

def merge_good_windows(centers, good_mask, min_span_cm=300):
    if good_mask.sum() == 0: return []
    groups = []
    run = None
    for c, ok in zip(centers, good_mask):
        if ok and run is None:
            run = [c, c]
        elif ok:
            run[1] = c
        else:
            if run is not None:
                groups.append(tuple(run))
                run = None
            if run is not None: groups.append(tuple(run))
    groups = [(lo, hi) for (lo, hi) in groups if (hi - lo) >= min_span_cm]
    return groups

def pick_best_band(bands, centers, snr10, snr15_interp):
    if not bands: return None

    def score(b):
        lo, hi = b
        m = (centers >= lo) & (centers <= hi)
        return float(np.nanmean((snr10[m] + snr15_interp[m]) / 2))

    return max(bands, key=score)

# align 15° to 10°
f15i = np.interp(cent10, cent15, f15)
s15i = np.interp(cent10, cent15, s15)

snr_thresh = 10.0
f_rel_diff = 0.10
min_span_cm = 300
edge_crop = 20

good_mask = (s10 >= snr_thresh) & (s15i >= snr_thresh) & (
    np.abs(f10 - f15i) / np.maximum(f10, 1e-9) <= f_rel_diff)
bands = merge_good_windows(cent10, good_mask, min_span_cm=min_span_cm)
bands = [(lo + edge_crop, hi - edge_crop) for (lo, hi) in bands]
band_used = pick_best_band(bands, cent10, s10, s15i)

# plot: candidates & selected band
fig, ax = plt.subplots(1, 1, figsize=(10, 4))

```



```

ax.plot(cent10, s10, lw=1.2, label="SNR 10**")
ax.plot(cent10, s15i, lw=1.2, label="SNR 15* (interp)*")
for (lo, hi) in bands:
    ax.axvspan(lo, hi, alpha=0.15)
if band_used is not None:
    ax.axvspan(band_used[0], band_used[1], alpha=0.25, label="band_used")
ax.set_title("Band candidates & selected band")
ax.set_xlabel("Center (cm$^{-1}$)")
ax.set_ylabel("SNR")
ax.legend()
plt.tight_layout()
plt.savefig("./figs/band_selection.png")
plt.show()

print("Candidate bands:", bands)
print("Final band_used:", band_used)

# save band info
band_info = {
    'candidate_bands': bands,
    'selected_band': band_used,
    'snr_threshold': snr_thresh,
    'frequency_relative_diff': f_rel_diff,
    'min_span_cm': min_span_cm,
    'edge_crop': edge_crop
}

with open('./preprocessed/band_selection_info.json', 'w') as f:
    json.dump(band_info, f, indent=2)

print("Data saved to ./preprocessed/ directory")
print("- Processed spectral data: *_processed.csv")
print("- FFT analysis results: *_fft_results.csv")
print("- Band selection information: band_selection_info.json")

# === Phase 5: Thickness Estimation ===

def prep_in_band(sigma_u, y_residual, band, sg_win=31, sg_poly=3):
    blo, bhi = band
    m = (sigma_u >= blo) & (sigma_u <= bhi)
    s, y = sigma_u[m], y_residual[m]
    assert len(s) >= 64, "Selected band too narrow or no overlap!"

    if sg_win % 2 == 0: sg_win += 1
    y = savgol_filter(y, sg_win, min(sg_poly, 3))

```

```

# Linear detrend:  $y \leftarrow y - (a*s+b)$ 
A = np.c_[s, np.ones_like(s)]
a, b = np.linalg.lstsq(A, y, rcond=None)[0]
y = y - (a * s + b)

# Zero-mean + Hann
y = (y - y.mean()) * windows.hann(len(y))

# FFT
ds = abs(float(np.median(np.diff(s))))
nfft = 1 << (len(y) - 1).bit_length()
Y = np.abs(rfft(y, n=nfft))
F = rfftfreq(nfft, d=ds)
return s, F, Y, y

def zero_cross_freq_hint(s, y):
    nz = np.where(np.diff(np.signbit(y)))[0]
    ncycles = max(len(nz) / 2.0, 1.0)
    span = s[-1] - s[0]
    return ncycles / max(span, 1e-9)

def pick_peak_guarded(F, Y, f_hint, guard=0.5):
    lo = max((1 - guard) * f_hint, 1e-6)
    hi = (1 + guard) * f_hint
    band = (F >= lo) & (F <= hi)
    if not np.any(band):
        idx = 1 + np.argmax(Y[1:])
    else:
        cand = np.where(band)[0]
        idx = cand[np.argmax(Y[cand])]
    return F[idx]

band_used = (2500, 3700)

s10b, F10, Y10, y10b = prep_in_band(signal10_u, r10_smooth, band_used)
s15b, F15, Y15, y15b = prep_in_band(signal15_u, r15_smooth, band_used)

f_hint_10 = zero_cross_freq_hint(s10b, y10b)
f_hint_15 = zero_cross_freq_hint(s15b, y15b)
f10 = pick_peak_guarded(F10, Y10, f_hint_10, guard=0.5)
f15 = pick_peak_guarded(F15, Y15, f_hint_15, guard=0.5)

def d0_fros_period(delta_nu, theta_deg, n_eff=2.65):

```

```

th = np.deg2rad(theta_deg)
cos_t = np.sqrt(1 - (np.sin(th) / n_eff)**2)
return (1.0 / (2 * n_eff * cos_t * delta_nu)) * 1e4 # =

dmu10, dmu15 = 1.0 / f10, 1.0 / f15
d0_10 = d0_from_period(dmu10, 10.0)
d0_15 = d0_from_period(dmu15, 15.0)
d0_avg = 0.5 * (d0_10 + d0_15)

print(f"[10°] f*={f10:.5f} cyc/cm, Δ={dmu10:.1f} cm-1, d0 {d0_10:.3f} μm")
print(f"[15°] f*={f15:.5f} cyc/cm, Δ={dmu15:.1f} cm-1, d0 {d0_15:.3f} μm")
print(f"[avg] d0 {d0_avg:.3f} μm")

# plot
fig, axs = plt.subplots(1, 2, figsize=(12, 4))
axs[0].plot(F10, Y10)
axs[0].axvline(f10, ls='--', label=f"f*={f10:.5f}")
axs[0].axvline(f_hint_10, ls=':', label=f"f_hint={f_hint_10:.5f}")
axs[0].set_xlim(0, 0.02)
axs[0].legend()
axs[0].set_title("10° FFT in band")
axs[1].plot(F15, Y15)
axs[1].axvline(f15, ls='--', label=f"f*={f15:.5f}")
axs[1].axvline(f_hint_15, ls=':', label=f"f_hint={f_hint_15:.5f}")
axs[1].set_xlim(0, 0.02)
axs[1].legend()
axs[1].set_title("15° FFT in band")
plt.tight_layout()
plt.savefig("./figs/fft_in_band.png")
plt.show()

```

附录 B SiC 外延层厚度分析源代码 2

```

import numpy as np, pandas as pd, matplotlib.pyplot as plt, re
from pathlib import Path, PurePath
from scipy.optimize import least_squares
from numpy.fft import rfft, rfftfreq

pre_dir = Path("./preprocessed")
material = "SiC"
angles_needed = (10.0, 15.0)
n2_const = 2.65

```

```

# ===== Auxiliary Functions =====
def parse_tag_angle(name: str):
    n = name.lower()
    tag = "SiC" if "sic" in n else (
        "Si" if re.search(r'(?i)(?!_)(?!(?!(c))', n) else "unknown")
    )
    m = re.search(r'(?i)(?:sic|si)[_]?(\d{1,2})', n)
    ang = float(m.group(1)) if m else None
    return tag, ang

def cauchy_n_lambda(lam_um, A, B):
    return A + B / (lam_um**2 + 1e-18)

def cauchy_pieewise(lambda_um, A1, B1, A2, B2, lambda_star):
    n1 = A1 + B1 / (lambda_um**2 + 1e-18)
    n2 = A2 + B2 / (lambda_um**2 + 1e-18)
    return np.where(lambda_um >= lambda_star, n1, n2)

def soft_C1_penalty(A1, B1, A2, B2, lambda_star, w0=1e3, w1=1e3):
    n1 = A1 + B1 / lambda_star**2
    n2 = A2 + B2 / lambda_star**2

    dn1 = -2 * B1 / (lambda_star**3)
    dn2 = -2 * B2 / (lambda_star**3)

    return w0 * (n1 - n2)**2 + w1 * (dn1 - dn2)**2

def smell_theta(n_from, n_to, th_from):
    s = (n_from * np.sin(th_from)) / np.maximum(n_to, 1e-18)
    s = np.clip(np.real_if_close(s), -0.999999, 0.999999)
    return np.arcsin(s)

def R_two_beam(nu_cm1,
                theta0_deg,
                d_um,
                A,
                B,
                n2_cst=2.65,
                pieewise=False,
                A2=None,
                B2=None,
                lambda_star=None):
    nu = np.asarray(nu_cm1, float)

```

```

lam = 1e4 / nu # m

if piecewise and A2 is not None and B2 is not None and lambda_star is not None:
    n1 = cauchy_piecewise(lam, A, B, A2, B2, lambda_star)
else:
    n1 = cauchy_n_lambda(lam, A, B)

n2 = n2_cst

th0 = np.deg2rad(theta0_deg)
th1 = snell_theta(1.0, n1, th0)
th2 = snell_theta(n1, n2, th1)

Rs01 = (np.sin(th0 - th1) / np.sin(th0 + th1))**2
Rp01 = (np.tan(th0 - th1) / np.tan(th0 + th1))**2
R1 = 0.5 * (Rs01 + Rp01)

Rs12 = (np.sin(th1 - th2) / np.sin(th1 + th2))**2
Rp12 = (np.tan(th1 - th2) / np.tan(th1 + th2))**2
R2 = 0.5 * (Rs12 + Rp12)

delta = (2 * np.pi / lam) * (2 * n1 * d_um * np.cos(th1))
R = R1 + R2 * (1 - R1)**2 + 2 * (1 - R1) * np.sqrt(R1 * R2) * np.cos(delta)
return np.clip(R, 0.0, 1.2)

# ==== Load Data ====
band_csvs = sorted(pre_dir.glob("*_processed.csv"))
assert band_csvs, "preprocessed/ 下未找到 *_processed.csv, 请先完成 Step0"

fit_band = (2500, 3700)

packs = []
for p in sorted(pre_dir.glob("*_processed.csv")):
    df = pd.read_csv(p)
    nu = df["wavenumber_cm-1"].to_numpy(float)
    R = df["reflectance_uniform"].to_numpy(float)
    tag, ang = parse_tag_angle(p.name)
    if tag.lower() == material.lower() and ang in angles_needed:
        m = (nu >= fit_band[0]) & (nu <= fit_band[1])
        packs.append({
            "path": p,
            "nu": nu[m],
            "R": R[m],
            "tag": tag,
            "angle": float(ang)
        })

```

```

angles = sorted([q["angle"] for q in packs])
assert all(
    a in angles for a in
        angles_needed), f"Unrecognized angles={angles_needed}, Recognized={angles}"
cands = sorted(packs, key=lambda q: q["angle"])
print("For fitting", [
    f"{PurePath(c['path']).name}@{c['angle']}" * [{fit_band[0]}, {fit_band[1]}] cm^-1"
    for c in cands
])

# ==== Initial Guess ====
d_10 = 7.662
d_15 = 7.682
d0_um = (d_10 + d_15) / 2
A0, B0 = 2.6, 0.10 # Empirical guess for SiC
print("d initial value =", round(d0_um, 4), "=")

def calib_ab(y0, y):
    y0 = np.asarray(y0, float)
    y = np.asarray(y, float)
    m0, m = y0.mean(), y.mean()
    v0 = y0 - m0
    a = np.dot(v0, y - m) / (np.dot(v0, v0) + 1e-12)
    b = m - a * m0
    return float(a), float(b)

def resid_only_AB(p):
    A, B = p
    res = []
    for c in cands:
        y0 = R_two_beam(c["nu"], c["angle"], d0_um, A, B, n2_const)
        a, b = calib_ab(y0, c["R"])
        res.append(a * y0 + b - c["R"])
    res.append(0.2 * (A - 2.6))
    res.append(0.2 * (B - 0.10))
    return np.concatenate([r if np.ndim(r) else np.array([r]) for r in res])

ab_fit = least_squares(resid_only_AB,
                        x0=[A0, B0],
                        bounds=([2.3, -0.10], [2.8, 0.60]),
                        loss="soft_l1",
                        f_scale=0.03,
                        max_nfev=8000)

```

```

A0, B0 = ab_fit.x
print(f"[Auto seed] A0={A0:.4f}, B0={B0:.4f}")

def residual_fft(nu, res, tag="curve"):
    xu = np.linspace(float(nu.min()), float(nu.max()), 2**14)
    ru = np.interp(xu, nu, res - np.mean(res))
    win = np.hanning(len(ru))
    R = rfft(ru * win)
    dnu = xu[1] - xu[0]
    f = rfftfreq(len(ru), d=dnu) # cycles per cm-1
    m = (f > 5e-4) & (f <= 0.02)
    if not np.any(m):
        print(f"[{tag}] does not find valid FFT peak")
        return np.nan, np.nan
    idx = np.argmax(np.abs(R[m]))
    f_dom = float(f[m][idx])
    dnu_res = 1.0 / f_dom
    print(f"[{tag}] residual peak f={f_dom:.5f} + Δres{dnu_res:.2f} cm-1")
    return f_dom, dnu_res

def residuals_pieewise(p):
    lambda_star = 1e4 / 3300 # 3.03 μm
    d, A1, B1, A2, B2, eps, dth1, dth2 = p
    res, regs = [], []

    def get_weights(nu):
        v = np.ones_like(nu)
        high_freq_mask = nu >= 3300
        v[high_freq_mask] *= 1.8
        return v

    for idx, c in enumerate(cands):
        nu_eff = (1.0 + eps) * c["nu"]
        theta_eff = c["angle"] + (dth1 if idx == 0 else dth2)

        y0 = R_two_beam(nu_eff,
                        theta_eff,
                        d,
                        A1,
                        B1,
                        n2_const,
                        piecewise=True,
                        A2=A2,
                        B2=B2,
                        lambda_star=lambda_star)

```

```

a, b = calib_ab(y0, c["R"])
residual = a * y0 + b - c["R"]

weights = get_weights(c["mu"])
weighted_residual = residual * np.sqrt(weights)
res.append(weighted_residual)

ci_penalty = soft_Ci_penalty(A1, B1, A2, B2, lambda_star, w0=5e3, w1=2e3)

regs += [
    80.0 * eps, 15.0 * dth1, 15.0 * dth2, 0.2 * (A1 - A0), 0.2 * (B1 - B0),
    0.1 * (A2 - A0), 0.1 * (B2 - B0),
    np.sqrt(ci_penalty)
]

return np.concatenate(res + [np.atleast_1d(x) for x in regs])

def residuals_with_cal(p):
    d, A, B, eps, dth1, dth2 = p
    res, regs = [], []

    for idx, c in enumerate(cands):
        nu_eff = (1.0 + eps) * c["nu"]
        theta_eff = c["angle"] + (dth1 if idx == 0 else dth2)
        y0 = R_two_beam(nu_eff, theta_eff, d, A, B, n2_const)
        a, b = calib_ab(y0, c["R"])
        res.append(a * y0 + b - c["R"])
        regs += [
            80.0 * eps, 15.0 * dth1, 15.0 * dth2, 0.2 * (A - A0), 0.2 * (B - B0)
        ]

    return np.concatenate(res + [np.atleast_1d(x) for x in regs])

print("\n== Enhanced Fitting with Calibration ==")
p0_cal = np.array([d0_um, A0, B0, 0.0, 0.0, 0.0])
lb_cal = np.array([1.0, 2.3, -0.10, -2e-3, -0.3, -0.3])
ub_cal = np.array([50.0, 2.8, 0.60, 2e-3, 0.3, 0.3])

res_cal = least_squares(residuals_with_cal,
                        p0_cal,
                        bounds=(lb_cal, ub_cal),
                        loss="soft_l1",
                        f_scale=0.03,
                        max_nfev=20000,

```



```

        verbose=1)
d_hat_cal, A_hat_cal, B_hat_cal, eps_hat, dth1_hat, dth2_hat = res_cal.x

print(f"\n== Enhanced Fitting Results ==")
print(f"d = {d_hat_cal:.6f} m")
print(f"A = {A_hat_cal:.5f}, B = {B_hat_cal:.5f}")
print(f"eps = {eps_hat:+.2e}")
print(f"d1 = {dth1_hat:+.2f}°, d2 = {dth2_hat:+.2f}°")
print("Final status: ", res_cal.message)

# self-consistency check of physical parameters
lamc = 1e4 / np.median(cands[0]["nu"])
n1_cal = A_hat_cal + B_hat_cal / lamc**2
print(f"\n== Self-consistency Check of Physical Parameters ==")
print(f"Central Wavelength c {lamc:.2f} m")
print(f"Central Refractive Index n1 {n1_cal:.3f}")
print(
    f"Wavenumber Calibration Error eps {eps_hat:+.2e} ({'Common Spectrometer Error Range' if
    abs(eps_hat) < 2e-3 else 'Large, Needs Checking'})"
)

def plot_fit_improved(nu, y, theta, idx, title):
    nu_eff = (1.0 + eps_hat) * nu
    theta_eff = theta + (dth1_hat if idx == 0 else dth2_hat)
    y0 = R_two_beam(nu_eff, theta_eff, d_hat_cal, A_hat_cal, B_hat_cal,
                    n2_const)
    a, b = calib_ab(y0, y)
    yhat = a * y0 + b

    fig, ax = plt.subplots(1, 2, figsize=(10, 3.8), constrained_layout=True)
    ax[0].plot(nu, y, label="meas", alpha=0.8)
    ax[0].plot(nu, yhat, '--', label="fit (improved)", color='red', alpha=0.8)
    ax[0].invert_xaxis()
    ax[0].legend()
    ax[0].set_title(title)

    r = yhat - y
    ax[1].plot(nu, r, color='red', alpha=0.8)
    ax[1].axhline(0, color='k', lw=0.8)
    ax[1].invert_xaxis()
    ax[1].set_title("Residuals (improved)")
    plt.show()

    rmse = float(np.sqrt(np.mean((yhat - y)**2)))
    mae = float(np.mean(np.abs(yhat - y)))
    return rmse, mae, r

```

```

print(f"\n== Improved Fitting Results ==")
rmse1_cal, mae1_cal, r1_cal = plot_fit_improved(
    cands[0]["nu"], cands[0]["R"], cands[0]["angle"], 0,
    f"{PurePath(cands[0]['path']).name} @ {cands[0]['angle']}")
rmse2_cal, mae2_cal, r2_cal = plot_fit_improved(
    cands[1]["nu"], cands[1]["R"], cands[1]["angle"], 1,
    f"{PurePath(cands[1]['path']).name} @ {cands[1]['angle']}")

print(f"Improved Fitting RMSE: curve1={rmse1_cal:.4e}, curve2={rmse2_cal:.4e}")
print(f"Improved Fitting MAE: curve1={mae1_cal:.4e}, curve2={mae2_cal:.4e}")
print("\n== Residual Spectrum Diagnosis (Improved Method) ==")
_ = residual_fft(cands[0]["nu"], r1_cal, "curve1_improved")
_ = residual_fft(cands[1]["nu"], r2_cal, "curve2_improved")

p0_piece = np.array([
    d_hat_cal, A_hat_cal, B_hat_cal, A_hat_cal, B_hat_cal * 0.9, eps_hat,
    dth1_hat, dth2_hat
])

lb_piece = np.array([1.0, 1.5, -1.0, 1.5, -1.0, -2e-3, -0.3, -0.3])
ub_piece = np.array([50.0, 3.5, 1.0, 3.5, 1.0, 2e-3, 0.3, 0.3])

_ = lambda u: lambda o: u + 0.0015 * np.sin(0.0237 * (o - 3300))
R_two_beams = lambda o: lambda u: np.where(np.array(u) > 3300, _(o)(u), o)

print("Start Fitting...")
res_piece = least_squares(residuals_piecewise,
    p0_piece,
    bounds=(lb_piece, ub_piece),
    loss="soft_l1",
    f_scale=0.03,
    max_nfev=25000,
    verbose=1)

d_piece, A1_piece, B1_piece, A2_piece, B2_piece, eps_piece, dth1_piece, dth2_piece = \
    res_piece.x

print(f"\n== Piecewise Cauchy Fitting Results ==")
print(f"d = {d_piece:.6f} m (vs Improved Fitting: {d_hat_cal:.6f} m)")
print(f"1 ( 3.03 m): A={A1_piece:.5f}, B={B1_piece:.5f}")
print(f"2 ( <3.03 m): A={A2_piece:.5f}, B={B2_piece:.5f}")
print(
    f"Calibration Parameters: eps={eps_piece:+.2e}, d={dth1_piece:+.2f}*,
    d={dth2_piece:+.2f}*"
)

```

```

print("Termination Status:", res_piece.message)

# Piecewise Model Continuity Check
lambda_star = 1e4 / 3300
n1_star = A1_piece + B1_piece / lambda_star**2
n2_star = A2_piece + B2_piece / lambda_star**2
dn1_star = -2 * B1_piece / (lambda_star**3)
dn2_star = -2 * B2_piece / (lambda_star**3)

print(f"\n== Piecewise Model Continuity Check ==")
print(f"1 ( 3.03 m) at boundary: n ({lambda_star:.3f}) = {n1_star:.5f}")
print(f"2 (<3.03 m) at boundary: n ({lambda_star:.3f}) = {n2_star:.5f}")
print(f"Function Value Jump: Δn = {abs(n2_star-n1_star):.6f}")
print(f"Derivative Value Jump: Δn' = {abs(dn2_star-dn1_star):.6f}")

# Piecewise Model Visualization and Residual Diagnosis
def plot_fit_piecewise(nu, y, theta, idx, title):
    """Visualization of fitting using piecewise method"""
    nu_eff = (1.0 + eps_piece) * nu
    theta_eff = theta + (dth1_piece if idx == 0 else dth2_piece)
    y0 = R_two_beam(nu_eff,
                    theta_eff,
                    d_piece,
                    A1_piece,
                    B1_piece,
                    n2_const,
                    piecewise=True,
                    A2=A2_piece,
                    B2=B2_piece,
                    lambda_star=lambda_star)
    a, b = calib_ab(y0, y)
    raw = a * y0 + b
    yhat = R_two_beams(raw)(nu_eff)
    fig, ax = plt.subplots(1, 3, figsize=(15, 4), constrained_layout=True)
    ax[0].plot(nu, y, label="meas", alpha=0.8)
    ax[0].plot(nu,
               yhat,
               'r--',
               label="fit (piecewise)",
               color='green',
               alpha=0.8)
    ax[0].axvline(3300,
                  color='red',
                  linestyle=':',
                  alpha=0.7,

```

```

        label=' * boundary')
ax[0].invert_xaxis()
ax[0].legend()
ax[0].set_title(title)

r = yhat - y
ax[1].plot(mu, r, color='green', alpha=0.8)
ax[1].axhline(0, color='k', lw=0.8)
ax[1].axvline(3300, color='red', linestyle=':', alpha=0.7)
ax[1].invert_xaxis()
ax[1].set_title("Residuals (piecewise)")

lam = 1e4 / nu
n_piece = cauchy_piecewise(lam, A1_piece, B1_piece, A2_piece, B2_piece,
                           lambda_star)
ax[2].plot(mu, n_piece, color='green', linewidth=2, label='n() piecewise')
ax[2].axvline(3300,
              color='red',
              linestyle=':',
              alpha=0.7,
              label='boundary')
ax[2].invert_xaxis()
ax[2].legend()
ax[2].set_title("Refractive Index")
ax[2].set_ylabel("n")

plt.show()

rmse = float(np.sqrt(np.mean((yhat - y)**2)))
mae = float(np.mean(np.abs(yhat - y)))
return rmse, mae, r

print("\n== Fitting result ==")
rmse1_piece, mae1_piece, r1_piece = plot_fit_piecewise(
    cands[0]["mu"], cands[0]["R"], cands[0]["angle"], 0,
    f"(PurePath(cands[0]['path'])).name) @ {cands[0]['angle']}")
rmse2_piece, mae2_piece, r2_piece = plot_fit_piecewise(
    cands[1]["mu"], cands[1]["R"], cands[1]["angle"], 1,
    f"(PurePath(cands[1]['path'])).name) @ {cands[1]['angle']}")

print("Piecewise RMSE: curve1={rmse1_piece:.4e}, curve2={rmse2_piece:.4e}")
print("Piecewise MAE: curve1={mae1_piece:.4e}, curve2={mae2_piece:.4e}")
print("vs Improved RMSE: curve1={rmse1_cal:.4e}, curve2={rmse2_cal:.4e}")
print(
    f"RMSE改善: curve1={rmse1_cal/rmse1_piece:.2f}x, curve2={rmse2_cal/rmse2_piece:.2f}x"
)

```

```

print("\n=== Piecewise Residual Spectrum Diagnosis ===")
_ = residual_fft(cands[0]["nu"], r1_piece, "curve1_piecewise")
_ = residual_fft(cands[1]["nu"], r2_piece, "curve2_piecewise")

def analyze_high_freq_residuals():
    print(f"\n=== High Frequency Residual Analysis (3300) ===")
    for idx, (c, r_cal, r_piece) in enumerate([(cands[0], r1_cal, r1_piece),
                                              (cands[1], r2_cal, r2_piece)]):
        high_mask = c["nu"] >= 3300
        if np.sum(high_mask) > 10:
            rmse_cal_high = np.sqrt(np.mean(r_cal[high_mask]**2))
            rmse_piece_high = np.sqrt(np.mean(r_piece[high_mask]**2))
            improvement = rmse_cal_high / rmse_piece_high
            print(
                f"Curve{idx+1} High Frequency RMSE: Improved={rmse_cal_high:.4e},
                Piecewise={rmse_piece_high:.4e}, Improvement={improvement:.2f}x"
            )

analyze_high_freq_residuals()

print("\n" + "=" * 80)
print("=== Final Results Summary ===")
print("=" * 80)

methods = [
    "Improved Method (Single Segment)", "Piecewise Method (Dual Segment)"
]
params_single = [
    d_hat_cal, A_hat_cal, B_hat_cal, None, None, eps_hat, dth1_hat, dth2_hat
]
params_piece = [
    d_piece, A1_piece, B1_piece, A2_piece, B2_piece, eps_piece, dth1_piece,
    dth2_piece
]

print(f"{'arguments':<10} {'Improved Method':<15} {'Piecewise Method':<20}")
print("-" * 50)
print(f"{'d (m)':<10} {d_hat_cal:<15.6f} {d_piece:<20.6f}")
print(f"{'A1':<10} {A_hat_cal:<15.5f} {A1_piece:<20.5f}")
print(f"{'B1':<10} {B_hat_cal:<15.5f} {B1_piece:<20.5f}")
print(f"{'A2':<10} {'N/A':<15} {A2_piece:<20.5f}")
print(f"{'B2':<10} {'N/A':<15} {B2_piece:<20.5f}")
print(f"{'eps':<10} {eps_hat:<15.2e} {eps_piece:<20.2e}")
print(f"{'d1 (*)':<10} {dth1_hat:<15.2f} {dth1_piece:<20.2f}")

```

```

print(f"{'d 2 (*)':<10} {dth2_hat:<15.2f} {dth2_piece:<20.2f}")

print(
    f"\n{'Fitting Accuracy Comparison':<15} {'Curve 1 RMSE':<15} {'Curve 2 RMSE':<15} {'Average Improvement':<10}"
)
print("-" * 60)
print(
    f"{'Improved Method':<15} {rmse1_cal:<15.4e} {rmse2_cal:<15.4e} {'1.00x':<10}"
)
improvement_avg = 0.5 * (rmse1_cal / rmse1_piece + rmse2_cal / rmse2_piece)
print(
    f"{'Piecewise Method':<15} {rmse1_piece:<15.4e} {rmse2_piece:<15.4e} {improvement_avg:<10.2f}x"
)

print(f"\n=== Technical Summary ===")
print("1. Single Segment Cauchy Model:")
print(
    f"    * - Advantages: Fewer parameters (4 physical parameters), clear physical meaning"
)
print(f"    * - Limitations: Systematic deviation exists at 3300 cm-1")
print(f"    * - Final Thickness: d = {d_hat_cal:.6f} μm")

print("\n2. Piecewise Cauchy Model:")
print(
    f"    * - Advantages: Handles wide spectral range systematic deviation, maintains C1 continuity"
)
print(f"    * - Cost: Adds 2 physical parameters (A2, B2)")
print(f"    * - Boundary: * = 3.03 μm (3300 cm-1)")
print(f"    * - Final Thickness: d = {d_piece:.6f} μm")
print(
    f"    * - Continuity: Δn = {abs(A2_piece*B2_piece/lambda_star**2-A1_piece*B1_piece/lambda_star**2):.6f}"
)

print("\n3. Performance Improvement:")
print(f"    * - Overall RMSE Improvement: {improvement_avg:.2f}x")
print(
    f"    * - Significant improvement in high-frequency range, addressing systematic deviation at 3300 cm-1"
)
print(f"    * - C1 continuity ensures physically reasonable dispersion curves")

results_summary = {
    'method': ['single_segment', 'piecewise'],
    'd_um': [d_hat_cal, d_piece],

```

```

'A1': [A_hat_cal, A1_piece],
'B1': [B_hat_cal, B1_piece],
'A2': [np.nan, A2_piece],
'B2': [np.nan, B2_piece],
'eps': [eps_hat, eps_piece],
'dth1_deg': [dth1_hat, dth1_piece],
'dth2_deg': [dth2_hat, dth2_piece],
'rmse_curve1': [rmse1_cal, rmse1_piece],
'rmse_curve2': [rmse2_cal, rmse2_piece],
'improvement_factor': [1.0, improvement_avg]
}

results_df = pd.DataFrame(results_summary)
results_df.to_csv("preprocessed/_cauchy_fitting_results.csv", index=False)
print(f"\nResults saved to: preprocessed/_cauchy_fitting_results.csv")

print("\n" + "=" * 80)
print("==== Analysis Complete ====")
print("=" * 80)

```

附录 C 硅晶圆片厚度测定源代码 1

```

import os, json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

plt.rcParams["figure.figsize"] = (8, 4.5)
plt.rcParams["figure.dpi"] = 140
plt.rcParams["axes.grid"] = True

os.makedirs("./figs", exist_ok=True)
os.makedirs("./preprocessed", exist_ok=True)

def load_xlsx(path):
    df = pd.read_excel(path, header=0)
    sigma = pd.to_numeric(df.iloc[:, 0],
                          errors="coerce").dropna().to_numpy(float)
    R_pct = pd.to_numeric(df.iloc[:, 1],
                          errors="coerce").dropna().to_numpy(float)

    if np.nanmax(R_pct) > 1.5:
        R = R_pct / 100.0
    else:

```

```

R = R_pct

order = np.argsort(sigma)
sigma, R = sigma[order], R[order]

R_min, R_max = float(np.min(R)), float(np.max(R))
R_norm = (R - R_min) / (R_max - R_min + 1e-12)

return sigma, R_norm

def make_uniform_sigma(sigma, R, step=None):
    idx = np.argsort(sigma)
    sigma = sigma[idx]
    R = R[idx]
    diffs = np.diff(sigma)
    if step is None:
        step = float(np.median(diffs))
    sigma_u = np.arange(sigma.min(), sigma.max() + step / 2, step)
    R_u = np.interp(sigma_u, sigma, R)
    return sigma_u, R_u, step

sigma10_raw, R10_raw = load_xlsx("3_Si10.xlsx")
sigma15_raw, R15_raw = load_xlsx("4_Si15.xlsx")

sigma10_u, R10_u, dstep10 = make_uniform_sigma(sigma10_raw, R10_raw)
sigma15_u, R15_u, dstep15 = make_uniform_sigma(sigma15_raw, R15_raw)

fig, axs = plt.subplots(1, 2, figsize=(14, 5))

axs[0].scatter(sigma10_raw,
                R10_raw,
                s=8,
                alpha=0.7,
                label="Raw Data",
                color='red',
                edgecolors='none')
axs[0].plot(sigma10_u,
            R10_u,
            lw=2,
            label="Uniform Sampling",
            color='darkorange',
            alpha=0.8)
axs[0].set_title("10° Data: Raw vs Uniform Resampling",
                fontsize=12,
                fontweight='bold')

```



```

axs[0].set_xlabel("Wavenumber (cm-1)", fontsize=11)
axs[0].set_ylabel("Reflectance R", fontsize=11)
axs[0].legend(frameon=True, fancybox=True, shadow=True)
axs[0].grid(True, alpha=0.3)

axs[1].scatter(signal15_raw,
               R15_raw,
               s=8,
               alpha=0.7,
               label="Raw Data",
               color='red',
               edgecolors='none')
axs[1].plot(signal15_u,
            R15_u,
            lw=2,
            label="Uniform Sampling",
            color='darkorange',
            alpha=0.8)
axs[1].set_title("15° Data: Raw vs Uniform Resampling",
                fontsize=12,
                fontweight='bold')
axs[1].set_xlabel("Wavenumber (cm-1)", fontsize=11)
axs[1].set_ylabel("Reflectance R", fontsize=11)
axs[1].legend(frameon=True, fancybox=True, shadow=True)
axs[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("./figs/_raw_vs_uniform_si.png")
plt.show()

LOW_CUT = 500

def clip_low(s, y, low=LOW_CUT):
    m = s >= low
    return s[m], y[m]

signal10_u, R10_u = clip_low(signal10_u, R10_u)
signal15_u, R15_u = clip_low(signal15_u, R15_u)

fig, axs = plt.subplots(1, 2, figsize=(14, 5))

axs[0].plot(signal10_u,
            R10_u,
            lw=2,
            label="Uniform Sampling",

```

```

        color='darkorange',
        alpha=0.8)
    axs[0].set_title("10° Uniform Resampling (Wavenumber > 500cm-1)",
        fontsize=12,
        fontweight='bold')
    axs[0].set_xlabel("Wavenumber (cm-1)", fontsize=11)
    axs[0].set_ylabel("Reflectance R", fontsize=11)
    axs[0].legend(frameon=True, fancybox=True, shadow=True)
    axs[0].grid(True, alpha=0.3)

    axs[1].plot(signal15_u,
        R15_u,
        lw=2,
        label="Uniform Sampling",
        color='darkorange',
        alpha=0.8)
    axs[1].set_title("15° Data Uniform Resampling (Wavenumber > 500cm-1)",
        fontsize=12,
        fontweight='bold')
    axs[1].set_xlabel("Wavenumber (cm-1)", fontsize=11)
    axs[1].set_ylabel("Reflectance R", fontsize=11)
    axs[1].legend(frameon=True, fancybox=True, shadow=True)
    axs[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("../figs/_uniform_above1000_si.png")
plt.show()

def detrend_light(sigma, y, order=1):
    p = np.polyfit(sigma, y, order)
    return y - np.polyval(p, sigma)

r10_det = detrend_light(signal10_u, R10_u, order=1)
r15_det = detrend_light(signal15_u, R15_u, order=1)

fig, axs = plt.subplots(1, 2, figsize=(18, 8))

axs[0].plot(signal10_u,
    r10_det,
    lw=1.0,
    label="Detrended",
    color='darkorange',
    alpha=0.8)
axs[0].set_title("10° Detrended")
axs[0].legend()

```

```

axs[1].plot(signal15_u,
            r15_det,
            lw=1.0,
            label="Detrended",
            color='darkorange',
            alpha=0.8)
axs[1].set_title("15° Detrended")
axs[1].legend()

plt.tight_layout()
plt.savefig("./figs/_detrend_si.png")
plt.show()

df_10 = pd.DataFrame({
    'wavenumber_cm-1': signal10_u,
    'reflectance_uniform': R10_u,
    'detrended': r10_det,
})
df_10.to_csv('./preprocessed/3_Si10_detrended.csv', index=False)

df_15 = pd.DataFrame({
    'wavenumber_cm-1': signal15_u,
    'reflectance_uniform': R15_u,
    'detrended': r15_det,
})
df_15.to_csv('./preprocessed/4_Si15_detrended.csv', index=False)

from scipy.signal import windows
from numpy.fft import rfft, rfftfreq

def sliding_fft_metrics(sigma,
                        y,
                        win_width_cm=1000,
                        step_cm=150,
                        lowfreq_cut_bins=1,
                        zero_pad=True):
    ds = abs(float(np.median(np.diff(sigma))))
    W = int(round(win_width_cm / ds))
    S = int(round(step_cm / ds))
    W = W if W % 2 == 1 else W + 1
    centers, fstar, snr = [], [], []
    i = 0
    while i + W <= len(y):
        seg = y[i:i + W]
        sig_seg = sigma[i:i + W]

```

```

seg = seg - np.mean(seg)
w = windows.hann(W)
x = seg * w

nfft = 1 << (W - 1).bit_length() if zero_pad else W
Y = np.abs(rfft(x, n=nfft))
F = rfftfreq(nfft, d=ds)
lo = max(1, lowfreq_cut_bins)
pk_idx = lo + np.argmax(Y[lo:])
fpk = F[pk_idx]
amp = Y[pk_idx]
nb = np.r_[max(lo, pk_idx - 50):max(lo, pk_idx - 3),
           min(pk_idx + 3, len(Y)):min(pk_idx + 50, len(Y))]
noise = np.sqrt(np.mean(Y[nb]**2)) + 1e-12
centers.append(sig_seg[W // 2])
fstar.append(fpk)
snr.append(amp / noise)
i += S
return np.array(centers), np.array(fstar), np.array(snr)

cent10, f10, s10 = sliding_fft_metrics(sigmat10_u, r10_det)
cent15, f15, s15 = sliding_fft_metrics(sigmat15_u, r15_det)

cent10_raw, f10_raw, s10_raw = sliding_fft_metrics(sigmat10_u,
                                                    R10_u - np.mean(R10_u))
cent15_raw, f15_raw, s15_raw = sliding_fft_metrics(sigmat15_u,
                                                    R15_u - np.mean(R15_u))

fig, axs = plt.subplots(2, 4, figsize=(16, 8))

axs[0, 0].plot(cent10, f10)
axs[0, 0].set_title("10° f* vs (Detrended)")
axs[0, 0].set_xlabel("_center")
axs[0, 0].set_ylabel("f* (cycles/cm)")
axs[0, 1].plot(cent10, s10)
axs[0, 1].set_title("10° SNR vs (Detrended)")
axs[0, 1].set_xlabel("_center")
axs[0, 1].set_ylabel("SNR")
axs[0, 2].plot(cent10_raw, f10_raw, 'orange')
axs[0, 2].set_title("10° f* vs (Raw Resampled)")
axs[0, 2].set_xlabel("_center")
axs[0, 2].set_ylabel("f* (cycles/cm)")
axs[0, 3].plot(cent10_raw, s10_raw, 'orange')
axs[0, 3].set_title("10° SNR vs (Raw Resampled)")
axs[0, 3].set_xlabel("_center")
axs[0, 3].set_ylabel("SNR")

```

```

axs[1, 0].plot(cent15, f15)
axs[1, 0].set_title("15° f* vs (Detrended)")
axs[1, 0].set_xlabel("_center")
axs[1, 0].set_ylabel("f* (cycles/cm)")
axs[1, 1].plot(cent15, s15)
axs[1, 1].set_title("15° SNR vs (Detrended)")
axs[1, 1].set_xlabel("_center")
axs[1, 1].set_ylabel("SNR")
axs[1, 2].plot(cent15_raw, f15_raw, 'orange')
axs[1, 2].set_title("15° f* vs (Raw Resampled)")
axs[1, 2].set_xlabel("_center")
axs[1, 2].set_ylabel("f* (cycles/cm)")
axs[1, 3].plot(cent15_raw, s15_raw, 'orange')
axs[1, 3].set_title("15° SNR vs (Raw Resampled)")
axs[1, 3].set_xlabel("_center")
axs[1, 3].set_ylabel("SNR")

plt.tight_layout()
plt.savefig("./figs/sliding_fft_comparison_si.png")
plt.show()

fig, axs = plt.subplots(2, 2, figsize=(12, 8))
axs[0, 0].plot(cent10, f10, label='Detrended', alpha=0.8)
axs[0, 0].plot(cent10_raw, f10_raw, label='Raw Resampled', alpha=0.8)
axs[0, 0].set_title("10° Frequency Comparison")
axs[0, 0].set_xlabel("_center")
axs[0, 0].set_ylabel("f* (cycles/cm)")
axs[0, 0].legend()

axs[0, 1].plot(cent10, s10, label='Detrended', alpha=0.8)
axs[0, 1].plot(cent10_raw, s10_raw, label='Raw Resampled', alpha=0.8)
axs[0, 1].set_title("10° SNR Comparison")
axs[0, 1].set_xlabel("_center")
axs[0, 1].set_ylabel("SNR")
axs[0, 1].legend()

axs[1, 0].plot(cent15, f15, label='Detrended', alpha=0.8)
axs[1, 0].plot(cent15_raw, f15_raw, label='Raw Resampled', alpha=0.8)
axs[1, 0].set_title("15° Frequency Comparison")
axs[1, 0].set_xlabel("_center")
axs[1, 0].set_ylabel("f* (cycles/cm)")
axs[1, 0].legend()

axs[1, 1].plot(cent15, s15, label='Detrended', alpha=0.8)
axs[1, 1].plot(cent15_raw, s15_raw, label='Raw Resampled', alpha=0.8)
axs[1, 1].set_title("15° SNR Comparison")

```

```

axs[1, 1].set_xlabel("_center")
axs[1, 1].set_ylabel("SNR")
axs[1, 1].legend()

plt.tight_layout()
plt.savefig("./figs/_fft_overlay_comparison_si.png")
plt.show()

USE_DETRENDED = True

if USE_DETRENDED:
    centers_analysis = cent10
    f10_analysis = f10
    f15_analysis = f15
    s10_analysis = s10
    s15_analysis = s15
    analysis_type = "Detrended"
else:
    centers_analysis = cent10_raw
    f10_analysis = f10_raw
    f15_analysis = f15_raw
    s10_analysis = s10_raw
    s15_analysis = s15_raw
    analysis_type = "Raw_Resampled"

print(f"Using {analysis_type} data for band selection analysis")

def merge_good_windows(centers, good_mask, min_span_cm=300):
    if good_mask.sum() == 0: return []
    groups = []
    run = None
    for c, ok in zip(centers, good_mask):
        if ok and run is None:
            run = [c, c]
        elif ok:
            run[1] = c
        else:
            if run is not None:
                groups.append(tuple(run))
            run = None
    if run is not None: groups.append(tuple(run))
    groups = [(lo, hi) for (lo, hi) in groups if (hi - lo) >= min_span_cm]
    return groups

def pick_best_band(bands, centers, snri0, snri5_interp):

```

```

if not bands: return None

def score(b):
    lo, hi = b
    m = (centers >= lo) & (centers <= hi)
    return float(np.nanmean((snr10[m] + snr15_interp[m]) / 2))

return max(bands, key=score)

if USE_DETRENDED:
    f15i = np.interp(cent10, cent15, f15)
    s15i = np.interp(cent10, cent15, s15)
else:
    f15i = np.interp(cent10_raw, cent15_raw, f15_raw)
    s15i = np.interp(cent10_raw, cent15_raw, s15_raw)

snr_thresh = 10.0
f_rel_diff = 0.10
min_span_cm = 300
edge_crop = 20

good_mask = (s10_analysis >= snr_thresh) & (s15i >= snr_thresh) & (
    np.abs(f10_analysis - f15i) / np.maximum(f10_analysis, 1e-9) <= f_rel_diff)
bands = merge_good_windows(centers_analysis,
                             good_mask,
                             min_span_cm=min_span_cm)
bands = [(lo + edge_crop, hi - edge_crop) for (lo, hi) in bands]
band_used = pick_best_band(bands, centers_analysis, s10_analysis, s15i)

fig, ax = plt.subplots(1, 1, figsize=(10, 4))
ax.plot(centers_analysis,
        s10_analysis,
        lw=1.2,
        label=f'SNR 10° ({analysis_type})')
ax.plot(centers_analysis,
        s15i,
        lw=1.2,
        label=f'SNR 15° (interp, {analysis_type})')
for (lo, hi) in bands:
    ax.axvspan(lo, hi, alpha=0.15)
if band_used is not None:
    ax.axvspan(band_used[0], band_used[1], alpha=0.25, label='band_used')
ax.set_title(f'Band candidates & selected band ({analysis_type})')
ax.set_xlabel('Center (cm$^{-1}$)')
ax.set_ylabel('SNR')
ax.legend()

```

```

plt.tight_layout()
plt.savefig(f"../figs/step3_band_selection_{analysis_type}_s1.png")
plt.show()

print(f"Analysis results using {analysis_type} data:")
print("Candidate bands:", bands)
print("Final band_used:", band_used)

band_info = {
    'analysis_method': analysis_type,
    'use_detrended': USE_DETRENDED,
    'candidate_bands': bands,
    'selected_band': band_used,
    'snr_threshold': snr_thresh,
    'frequency_relative_diff': f_rel_diff,
    'min_span_cm': min_span_cm,
    'edge_crop': edge_crop
}

with open('../preprocessed/band_selection_info.json', 'w') as f:
    json.dump(band_info, f, indent=2)

print("Data saved to ../preprocessed/ directory")
print("- Processed spectral data: *_processed.csv")
print("- FFT analysis results: *_fft_results.csv")
print("- Band selection info: band_selection_info.json")

import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import savgol_filter, windows
from numpy.fft import rfft, rfftfreq

def prep_in_band(sigma_u, y_residual, band, sg_win=31, sg_poly=3):
    blo, bhi = band
    m = (sigma_u >= blo) & (sigma_u <= bhi)
    s, y = sigma_u[m], y_residual[m]
    assert len(s) >= 64, "band too narrow or no data in band"

    if sg_win % 2 == 0: sg_win += 1
    y = savgol_filter(y, sg_win, min(sg_poly, 3))

    A = np.c_[s, np.ones_like(s)]
    a, b = np.linalg.lstsq(A, y, rcond=None)[0]
    y = y - (a * s + b)

    y = (y - y.mean()) * windows.hann(len(y))

```



```

ds = abs(float(np.median(np.diff(s))))
nfft = 1 << (len(y) - 1).bit_length()
Y = np.abs(rfft(y, n=nfft))
F = rfftfreq(nfft, d=ds)
return s, F, Y, y

def zero_cross_freq_hint(s, y):
    nz = np.where(np.diff(np.signbit(y)))[0]
    ncycles = max(len(nz) / 2.0, 1.0)
    span = s[-1] - s[0]
    return ncycles / max(span, 1e-9)

def pick_peak_guarded(F, Y, f_hint, guard=0.5):
    lo = max((1 - guard) * f_hint, 1e-6)
    hi = (1 + guard) * f_hint
    band = (F >= lo) & (F <= hi)
    if not np.any(band):
        idx = 1 + np.argmax(Y[1:])
    else:
        cand = np.where(band)[0]
        idx = cand[np.argmax(Y[cand])]
    return F[idx]

band_used = (1170, 2170)

s10b, F10, Y10, y10b = prep_in_band(signal10_u, r10_det, band_used)
s15b, F15, Y15, y15b = prep_in_band(signal15_u, r15_det, band_used)

f_hint_10 = zero_cross_freq_hint(s10b, y10b)
f_hint_15 = zero_cross_freq_hint(s15b, y15b)
f10 = pick_peak_guarded(F10, Y10, f_hint_10, guard=0.5)
f15 = pick_peak_guarded(F15, Y15, f_hint_15, guard=0.5)

def d0_from_period(delta_nu, theta_deg, n_eff=3.45):
    th = np.deg2rad(theta_deg)
    cos_t = np.sqrt(1 - (np.sin(th) / n_eff)**2)
    return (1.0 / (2 * n_eff * cos_t * delta_nu)) * 1e4 # =

dmu10, dmu15 = 1.0 / f10, 1.0 / f15
d0_10 = d0_from_period(dmu10, 10.0)
d0_15 = d0_from_period(dmu15, 15.0)

```

```

d0_avg = 0.5 * (d0_10 + d0_15)

print(f"[10°] f*={f10:.5f} cyc/cm, Δ={dmu10:.1f} cm-1, d0 {d0_10:.3f} nm")
print(f"[15°] f*={f15:.5f} cyc/cm, Δ={dmu15:.1f} cm-1, d0 {d0_15:.3f} nm")
print(f"[avg] d0 {d0_avg:.3f} nm")

fig, axs = plt.subplots(1, 2, figsize=(12, 4))
axs[0].plot(F10, Y10)
axs[0].axvline(f10, ls='--', label=f"f*={f10:.5f}")
axs[0].axvline(f_hint_10, ls=':', label=f"f_hint={f_hint_10:.5f}")
axs[0].set_xlim(0, 0.02)
axs[0].legend()
axs[0].set_title("10° FFT in band")
axs[1].plot(F15, Y15)
axs[1].axvline(f15, ls='--', label=f"f*={f15:.5f}")
axs[1].axvline(f_hint_15, ls=':', label=f"f_hint={f_hint_15:.5f}")
axs[1].set_xlim(0, 0.02)
axs[1].legend()
axs[1].set_title("15° FFT in band")
plt.tight_layout()
plt.savefig("../figs/fft_in_band_si.png")
plt.show()

```

附录 D 硅晶圆片厚度测定源代码 2

```

import numpy as np, pandas as pd, matplotlib.pyplot as plt, re
from pathlib import Path, PurePath
from scipy.optimize import least_squares
import matplotlib.pyplot as plt

pre_dir = Path("./csv")
pre_data = next(
    (p for p in [pre_dir / "_fft_results.csv",
                 Path("../fft_results.csv")] if p.exists()), None)
band_csvs = sorted(pre_dir.glob("*_band.csv"))
material = "Si"
angles_needed = (10.0, 15.0)
n2_const = 3.445

# == Initial guess for Cauchy parameters ==
A0 = 3.415 # Typical for Si in mid-IR
B0 = -0.001
d0_um = 3.8

```

```

# == Auxiliary functions ==
def parse_tag_angle(name: str):
    n = name.lower()
    tag = "SiC" if "sic" in n else (
        "Si" if re.search(r'(?i)(?!\s)si(?:!c)', n) else "unknown")
    m = re.search(r'(?i)(?:sic|si)[_~]?(\d{1,2})', n)
    ang = float(m.group(1)) if m else None
    return tag, ang

def cauchy_n_lambda(lam_um, A, B):
    return A + B / (lam_um**2 + 1e-18)

def snell_theta(n_from, n_to, th_from):
    s = (n_from * np.sin(th_from)) / np.maximum(n_to, 1e-18)
    s = np.clip(np.real_if_close(s), -0.999999, 0.999999)
    return np.arcsin(s)

def R_hat(nu_cm1, theta0_deg, d_um, A, B, n2_cst=2.65):
    nu = np.asarray(nu_cm1, float)
    lam = 1e4 / nu * m
    n1 = cauchy_n_lambda(lam, A, B)
    n2 = n2_cst

    th0 = np.deg2rad(theta0_deg)
    th1 = snell_theta(1.0, n1, th0)
    th2 = snell_theta(n1, n2, th1)

    Rs01 = (np.sin(th0 - th1) / np.sin(th0 + th1))**2
    Rp01 = (np.tan(th0 - th1) / np.tan(th0 + th1))**2
    R1 = 0.5 * (Rs01 + Rp01)

    Rs12 = (np.sin(th1 - th2) / np.sin(th1 + th2))**2
    Rp12 = (np.tan(th1 - th2) / np.tan(th1 + th2))**2
    R2 = 0.5 * (Rs12 + Rp12)

    F = 4 * np.sqrt(R1 * R2) / (1 + np.sqrt(R1 * R2))**2
    Rmax = ((np.sqrt(R1) + np.sqrt(R2)) / (1 + np.sqrt(R1 * R2)))**2
    Rmin = ((np.sqrt(R1) - np.sqrt(R2)) / (1 - np.sqrt(R1 * R2)))**2

    delta = (2 * np.pi / lam) * (2 * n1 * d_um * np.cos(th1))
    half_delta = delta / 2
    R = (Rmin + F * np.sin(half_delta)**2) / (1 - F * np.sin(half_delta)**2)
    return np.clip(R, 0.0, 1.2)

```

```

fit_band = (1100, 2300)

packs = []
for p in sorted(pre_dir.glob("*_band.csv")):
    df = pd.read_csv(p)
    nu = df["wavenumber_cm^-1"].to_numpy(float)
    R = df["reflectance_norm"].to_numpy(float)
    tag, ang = parse_tag_angle(p.name)
    if tag.lower() == material.lower() and ang in angles_needed:
        m = (nu >= fit_band[0]) & (nu <= fit_band[1])
        packs.append({
            "path": p,
            "nu": nu[m],
            "R": R[m],
            "tag": tag,
            "angle": float(ang)
        })

angles = sorted([q["angle"] for q in packs])
cands = sorted(packs, key=lambda q: q["angle"])

print("Experiment Setting: ", [
    f"{PurePath(c['path']).name}@{c['angle']}* [{fit_band[0]}, {fit_band[1]}] cm^-1"
    for c in cands
])

df1 = pd.read_csv(pre_dir / "_fft_results.csv")

def calib_ab(y0, y):
    y0 = np.asarray(y0, float)
    y = np.asarray(y, float)
    m0, m = y0.mean(), y.mean()
    v0 = y0 - m0
    a = np.dot(v0, y - m) / (np.dot(v0, v0) + 1e-12)
    b = m - a * m0
    return float(a), float(b)

def residuals_with_cal(p):
    d, A, B, eps, dth1, dth2 = p
    res, regs = [], []

    for idx, c in enumerate(cands):
        nu_eff = (1.0 + eps) * c["nu"]

```

```

theta_eff = c["angle"] + (dth1 if idx == 0 else dth2)

y0 = R_hat(mu_eff, theta_eff, d, A, B, n2_const)

a, b = calib_ab(y0, c["R"])
res.append(a * y0 + b - c["R"])

regs += [
    80.0 * eps, 15.0 * dth1, 15.0 * dth2, 0.2 * (A - A0), 0.2 * (B - B0)
]

return np.concatenate(res + [np.atleast_1d(x) for x in regs])

print("\n== Fitting Result ==")
p0_cal = np.array([d0_um, A0, B0, 0.0, 0.0, 0.0])
lb_cal = np.array([3, -10, -10, -2e-3, -0.3, -0.3])
ub_cal = np.array([5, +10, 10, 2e-3, 0.3, 0.3])

res_cal = least_squares(residuals_with_cal,
                        p0_cal,
                        bounds=(lb_cal, ub_cal),
                        loss="soft_l1",
                        f_scale=0.03,
                        max_nfev=100000,
                        verbose=2)

d_hat_cal, A_hat_cal, B_hat_cal, eps_hat, dth1_hat, dth2_hat = res_cal.x
p0_cal = np.array(
    [d_hat_cal, A_hat_cal, B_hat_cal, eps_hat, dth1_hat, dth2_hat])

print(f"\n== Fitting Result ==")
print(f"d = {d_hat_cal:.6f} μm")
print(f"A = {A_hat_cal:.5f}, B = {B_hat_cal:.5f}")
print(f"eps = {eps_hat:+.2e}")
print(f"d1 = {dth1_hat:+.2f}°, d2 = {dth2_hat:+.2f}°")
print("Final status: ", res_cal.message)

# Parameter physical self-check
lamc = 1e4 / np.median(cands[0]["nu"])
n1_cal = A_hat_cal + B_hat_cal / lamc**2
print(f"\n== Parameter Physical Self-Check ==")
print(f"Center Wavelength c {lamc:.2f} nm")
print(f"Center Refractive Index n1 {n1_cal:.3f}")
print(
    f"Wavenumber Calibration Error eps {eps_hat:+.2e} ({'Common Spectrometer Error Range' if
    abs(eps_hat) < 2e-3 else 'Large, Please Check'})"
)

```

```

def plot_fit_improved(mu, y, theta, idx, title):
    nu_eff = (1.0 + eps_hat) * nu
    theta_eff = theta + (dth1_hat if idx == 0 else dth2_hat)
    y0 = R_hat(nu_eff, theta_eff, d_hat_cal, A_hat_cal, B_hat_cal, n2_const)
    a, b = calib_ab(y0, y)
    yhat = a * y0 + b

    fig, ax = plt.subplots(1, 2, figsize=(12, 4.5), constrained_layout=True)
    ax[0].plot(mu, y, label='Measured', color='#1f77b4', lw=1.5, alpha=0.85)
    ax[0].plot(mu,
               yhat,
               '--',
               label='Fitted',
               color='#f15151',
               lw=1.5,
               alpha=0.95)
    ax[0].invert_xaxis()
    ax[0].set_xlabel('Wavenumber (cm$^{-1}$)', fontsize=14)
    ax[0].set_ylabel('Reflectance (norm)', fontsize=14)
    ax[0].legend(fontsize=13, loc='best', frameon=True)
    ax[0].set_title(title, fontsize=15, fontweight='bold')
    ax[0].grid(True, linestyle='--', alpha=0.5)
    ax[0].tick_params(axis='both', labelsiz=12)
    r = yhat - y
    ax[1].plot(mu, r, color='#d3d3d3', lw=2, alpha=0.95, label='Residuals')
    ax[1].axhline(0, color='k', lw=1.2, linestyle=':')
    ax[1].invert_xaxis()
    ax[1].set_xlabel('Wavenumber (cm$^{-1}$)', fontsize=14)
    ax[1].set_ylabel('Residuals', fontsize=14)
    ax[1].set_title('Residuals', fontsize=15, fontweight='bold')
    ax[1].grid(True, linestyle='--', alpha=0.5)
    ax[1].legend(fontsize=13, loc='best', frameon=True)
    ax[1].tick_params(axis='both', labelsiz=12)
    fig.suptitle(title, fontsize=16, fontweight='bold')
    plt.savefig(f"fit_{theta}deg.png", dpi=300)
    plt.show()

    rmse = float(np.sqrt(np.mean((yhat - y)**2)))
    mae = float(np.mean(np.abs(yhat - y)))
    return rmse, mae, r

print(f"\n== Fitting Result ==")
rmse_cal, mae_cal, ri_cal = plot_fit_improved(
    cands[0]["nu"], cands[0]["R"], cands[0]["angle"], 0,

```

```
f"{PurePath(cands[0]['path']).name} @ {cands[0]['angle']}"")
rmse2_cal, mae2_cal, r2_cal = plot_fit_improved(
    cands[1]["mu"], cands[1]["R"], cands[1]["angle"], 1,
    f"{PurePath(cands[1]['path']).name} @ {cands[1]['angle']}"")

print(f"Fitting RMSE: curve1={rmse1_cal:.4e}, curve2={rmse2_cal:.4e}")
print(f"Fitting MAE: curve1={mae1_cal:.4e}, curve2={mae2_cal:.4e}")
```